Arhitectura Sistemelor de Calcul
Laborator - Partea 0x00

Bogdan Macovei
Ruxandra Bălucea
Cristian Rusu

Octombrie 2023

Cuprins

1 Introducere & Administrativ **3**
1.1 Introducere . . . 3
1.2 Administrativ . . . 4

2 Masina virtuala & Git **5**
2.1 Masina virtuala . . . 5
2.2 Git . . . 5
2.3 Mediul de lucru . . . 6

3 Limbajul Assembly x86 **7**
3.1 Registrii arhitecturii x86 . . . 9
3.1.1 Registrii de uz general . . . 9
3.1.2 EFLAGS . . . 10
3.1.3 Adrese . . . 10
3.1.4 Notatie . . . 10
3.2 Tipurile de date si structura de baza a unui program . . . 11
3.2.1 Tipurile de date . . . 11
3.2.2 Structura unui program . . . 11
3.3 Exemplu de program in x86 . . . 12
3.3.1 Instructiunea MOV . . . 12
3.3.2 Intreruperi de sistem . . . 12
3.3.3 Un prim program: cod sursa . . . 13
3.4 Programe in ASM si Debug . . . 15
3.4.1 Un alt program . . . 15
3.5 Hello, world! in Assembly . . . 17
3.6 Operatii aritmetice si logice . . . 18
3.6.1 Operatii aritmetice . . . 18
3.6.2 Exemplu . . . 18
3.6.3 Operatii logice . . . 19
3.6.4 Operatii de shift (deplasare) logice . . . 20
3.7 Salturi conditionate si neconditionate . . . 21

3.7.1 Salturile neconditionate . . . 21
3.7.2 Salturi conditionate . . . 21
3.7.3 Simularea structurilor repetitive . . . 23
3.7.4 Simularea structurilor repetitive: solutie 1 . . . 23
3.7.5 Simularea structurilor repetitive: solutie 2 . . . 24
3.8 Tablouri unidimensionale . . . 26
3.8.1 Adresarea in memorie . . . 26
3.8.2 Load Effective Address (LEA) . . . 26
3.8.3 Reprezentarea in memorie a tablourilor unidimensionale . . . 26
3.8.4 Declararea tablourilor unidimensionale . . . 27
3.8.5 Exemple: programul 1 . . . 27
3.8.6 Exemple: programul 2 . . . 27
3.9 Despre inline Assembly . . . 29

4 Exercitii propuse **30**
4.1 Laboratorul 1 . . . 30
4.2 Laboratorul 2 . . . 30
4.3 Laboratorul 3 . . . 30

5 Exercitii suplimentare **31**

6 Resurse suplimentare **33**

1 Introducere & Administrativ

1.1 Introducere

Scopul acestui laborator este acela de a va introduce limbajul Assembly x86 si de a va obisnui cu sintaxa AT&T pentru procesoarele x86. In acest sens, accentul este pus, in mare parte, pe insusirea notiunilor elementare de programare intr-un limbaj de asamblare.

Laboratorului va aplica in practica multe dintre principiile arhitecturii calculatoarelor descrise la curs si seminar. Ca programatori, exista multe avantaje daca intelegeti limbajul Assembly:

- veti intelege modul exact in care un calculator executa programele;
- veti putea optimiza la nivel de instructiune codul pe care il scrieti, cand aveti nevoie;
- veti putea intelege ce fac compilatoarele cu codul vostru sursa scris intr-un limbaj de nivel inalt (high level programming language);
- veti intelege modul in care sistemul de operare interactioneaza cu un program scris de voi (executia si oprirea programului, lucrul cu dispozitivele Input/Output prin sistemul de operare);
- veti intelege modul in care sistemul de operare interactioneaza cu hardware-ul vostru (veti putea scriere drivere);
- veti putea integra instructiuni inline-assembly in coduri de C/C++ pentru eficienta si control (veti putea programa sisteme de operare in timp real si sisteme de operare pentru microcontrollere);
- veti putea intelege modul in care functioneaza alte programe pentru care nu aveti disponibil codul sursa (dezasamblarea fisierelor binare uzuale, analiza si diagnoza programelor cu potential risc - virusi, malware, etc.).

Este important sa intelegem si care sunt dezavantajele limbajului Assembly:

- programele scrise in Assembly se pot rula doar pe arhitectura pentru care au fost scrise (machine-specific code), de exemplu programe scrise pentru x86 nu vor rula pe ARM-uri;
- e nevoie de un efort mult mai mare pentru a scrie un program in Assembly (in comparatie cu C/C++, sa nu mai vorbim de Java/C#/python);
- depanarea unui program Assembly mare (multe linii de cod) este extrem de dificila.

Trebuie sa tineti cont de aceste aspect in momentul in care alegeti limbajul de programare in care sa lucrati pentru un anumit proiect.

In acest laborator, cu toate ca accentul va fi pus pe scrierea de la zero a fisierelor .asm, sunt momente in care, pentru optimizare si pentru usurinta citirii codului, vom utiliza instructiuni scrise in limbaj de asamblare doar in anumite portiuni din codurile C/C++.

1.2 Administrativ

Modul de parcurgere saptamanal al laboratoarelor este descris pe pagina web a cursului iar detaliile cu privire la punctaj si conditiile de promovare sunt date la primul curs.

Prezenta la laborator nu se contorizeaza, astfel ca nu afecteaza in niciun mod rezultatul final. De asemenea, nici probleme din sectiunea Exercitii propuse nu se contorizeaza si nu influenteaza nota finala, dar este recomandat sa le lucrati. Testul final de laborator, probleme de la curs si de la examenul final vor fi in stilul problemelor descrise la seminar si laborator.

2 Masina virtuala & Git

Inainte de a incepe programarea in limbajul Assembly trebuie sa ne asiguram ca aveti instalat un sistem de operare corespunzator si aveti ustensilele de programare adecvate pentru scopul nostru.

2.1 Masina virtuala

Acest laborator va fi predat sub Linux, astfel ca daca nu aveti o distributie de Linux ca sistem principal de operare, va trebui sa va instalati cel putin pe o masina virtuala. Recomandarea este sa va luati o imagine de Ubuntu si sa o instalati pe VMWare[1] sau pe VirtualBox[2]. Imaginea sistemului de operare o gasiti pe site-ul de la Ubuntu `https://ubuntu.com/download/desktop`.

Un tutorial video excelent pentru instalarea VirtualBox impreuna cu unele sugestii de setare pentru sistem le gasiti online[3]. Daca nu ati mai folosit Linux, o investitie buna pentru viitorul vostru este un alt tutorial *putin* mai detaliat[4].

Pentru cei dintre voi care nu aveti sisteme de calcul x86 (de exemplu Mac M1/M2/etc.) urmati tutorialul de aici[5]. Veti avea nevoie sa instalati UTM[6] si veti avea nevoie sa emulati un Ubuntu x86, il gasiti in galeria UTM, dupa ce instalati UTM (alocati cel putin 4-6 GB RAM). Daca UTM nu functioneaza, puteti incerca si OrbStack[7].

2.2 Git

La aceast curs, tot codul sursa scris de voi va fi integrat intr-un sistem pentru controlul versiunilor. Aceasta este o practica standard in industria software. Munca din timpul laboratoarelor va fi salvata pe Git. In acest sens, va recomand sa va faceti un cont pe Github, sa va creati un repository cu numele "Laborator ASC" sau similar, iar toate problemele pe care le lucrati sa le aveti centralizate aici. Cand/daca aveti intrebari legate de programele realizate la acest laborator noi le putem verifica direct online pe Git si va putem sugera solutii.

Un tutorial video excelent pentru instalarea Git pe sistemul vostru este disponibil online[8].

Dupa ce sistemul Git este instalat, suntem gata sa scriem si salvam primul program de test. In terminal, vom scrie

```
mkdir Laborator1
cd Laborator1
git init

git config user.name "Nume Prenume"
git config user.email "email"
```

Vom vrea un fisier de baza in limbajul C,

[1] VMWare, `https://www.vmware.com/products/workstation-player.html`
[2] VirtualBox, `https://www.virtualbox.org/`
[3] How to Use VirtualBox (Beginners Guide), `https://www.youtube.com/watch?v=sB_5fqiysi4`
[4] The Complete Linux Course: Beginner to Power User!, `https://www.youtube.com/watch?v=wBp0Rb-ZJak`
[5] https://linuxhint.com/run-linux-m1-based-mac/
[6] https://mac.getutm.app/
[7] https://docs.orbstack.dev/machines/#intel-x86-emulation
[8] Git & GitHub Tutorial for Beginners, `https://www.youtube.com/playlist?list=PL4cUxeGkcC9goXbgTDQ0n_4TBzOO0ocPR`

```
nano file.c
```

In editorul deschis vom scrie primul program C

```c
#include <stdio.h>

int main()
{
    printf("Hello, world!\n");
    return 0;
}
```

Salvam: CTRL+X, Y, Enter.

In continuare, adaugam fisierul pe git si facem commit:

```
git add file.c
git commit -m 'primul commit pe acest proiect'
```

Din browser, salvati linkul cu numele git-ului, trebuie sa fie de forma `https://github.com/nume/numerepo.git`.

```
git remote add origin https://github.com/nume/numerepo.git
git push origin
```

In acest moment aveti fisierul salvat pe git. Verificati online ca fisierul sursa este vizibil.

2.3 Mediul de lucru

La acest curs, la cursuri urmatoare dar si in cariera voastra de programatori, va veti lovi de necesitatea de a folosi un sistem Linux si un sistem de control al versiunilor pentru produse software (de obicei Git). In consecinta, va rugam sa alocati timpul necesar pentru a va instala un mediu de lucru corespunzator unui dezvoltator de software. Trebuie sa aveti un anumit confort in instalarea acestor sisteme pentru ca le veti folosi in continuu de acum incolo, daca alegeti o cariera in IT.

In ultima instanta, daca nu reusiti sa instalati Linux si Git, va recomandam un sistem de programare Assembly avansat in browser[9].

[9] `https://www.onlinegdb.com/` sau `https://ideone.com/`

3 Limbajul Assembly x86

In acest capitol vom acoperi cele mai importante concepte de baza ale limbajului Assembly x86.

x86 este o familie de seturi de instructiuni introdusa o data cu microprocesoarele Intel 8086 (de aici porneste "86" din x86) si 8088. Setul de instructiuni[10] a fost adoptat de alti producatori de microprocesoare (notam in special contributia AMD) si astazi este standardul pentru microprocesoarele pe 64 de biti ale calculatoarelor personale. De-a lungul timpului setul de instructiuni a suferit multe modificari, dar mereu tinand cont de compatibilitatea cu arhitecturile x86 de calcul precedente. Notam aici o diferenta important: telefoanele inteligente si tabletele implementeaza o alta arhitectura de calcul *Advanced RISC Machine (ARM)*, in general.

Assembly x86 este un limbaj de programare de nivel scazut (low-level programming language) care permite controlul direct asupra unitatii centrale de calcul (CPU) a unui calculator cu arhitectura de calcul x86. In multe limbaje de programare de nivel inalt compilatorul creeaza prima data cod assembly ca un pas intermediar. Codul sursa assembly este o bucata de text, deci un procesor nu are cum sa interpreteze si sa execute acest text. Assembly este transformat apoi in cod masina (machine code): instructiuni masina binare codate clar (opcode) care pot fi apoi executate de procesor pas cu pas. Vrem sa va atragem atentia ca Assembly x86 are doua sintaxe distincte: Intel (folosit pe sisteme Windows) si AT&T (folosit predominant pe sisteme Unix). Diferentele intre cele doua nu sunt semnificative (este vorba doar de o sintaxa putin diferita) dar le puteti consulta online[11]. In acest laborator folosim doar sintaxa AT&T. Daca intelegeti una dintre sintaxe, cealalta e foarte usor de citit si inteles.

Componentele principale ale limbajului Assembly x86 sunt:

- Registrii. Ca in orice limbaj de programare, avem nevoie de variable cu care sa lucram. Aceste variable stocheaza date (in cazul nostru, valori pe 32 biti). Putem seta si modifica valorile registrilor si putem realiza operatii cu si intre acesti registri (de exemplu: putem aduna valorile din doi registri si il putem pune intr-un al treilea, sau in unul dintre registrii care deja stocheaza un operand). Unii registri stocheaza o locatie in memorie (de exemplu: trebuie sa stim unde se afla urmatoarea instructiune din program care trebuie executata, deci avem un Instruction Pointer - e defapt un "Instruction Register");
- Instructiuni. Procesorul suporta o serie de operatii intre registrii si locatii din memorie. In primul rand avem nevoie de abilitatea de a muta din/in registri date in/din memorie. In registri, putem face operatii matematice, logice, I/O, etc.
- Flaguri. Dupa ce facem o operatie, avem o serie de flag-uri (indicatori) care se activeaza in functie de rezultat (daca rezultatul operatilor este negativ sau zero, sau daca un overflow s-a produs);
- Adresarea memoriei. In procesor avem doar cativa registri. Daca avem nevoie sa lucram cu un volum mare de date (de exemplu, continutul unui fisier) atunci avem nevoie sa ne putem referi la o adresa din memorie (*Random Access Memory - RAM*). Accesul la registri este rapid, dar citirea/scrierea datelor din/in RAM este o operatie mult mai lenta ca timp;

[10] Intel® 64 and IA-32 Architectures Software Developer Manuals, `http://www.intel.com/content/www/us/en/processors/architectures-software-developer-manuals.html`

[11] `http://asm.sourceforge.net/articles/linasm.html#Syntax`

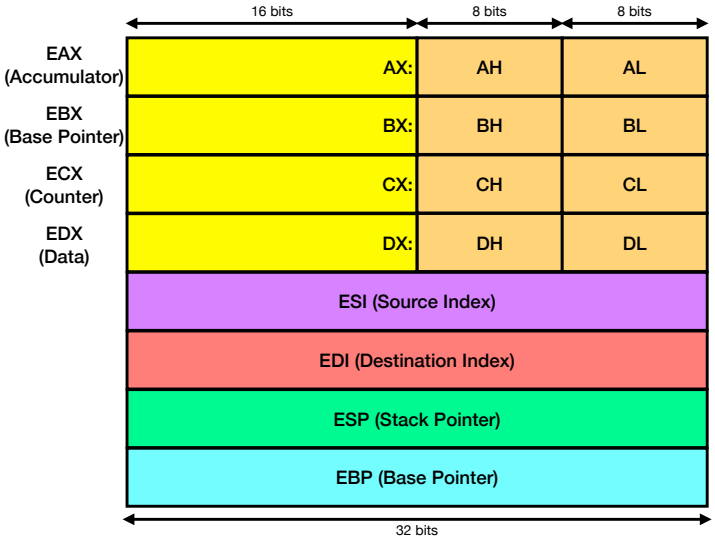
- Stiva programului. Orice program care se afla in executie are o locatie speciala de memorie care se numeste stiva (*the program stack*). Aceasta este o structura de date de tip *Last In First Out (LIFO)*. Vom vedea mai tarziu exemple unde aceasta structura este folositoare (apelul de functii in Assembly);

- Intreruperi. Programul pe care voi il executati (indiferent de limbajul de programare) nu are acces direct la toate resursele hardware. In cele mai multe cazuri, sistemul de operare controleaza accesul la memorie si periferice. De aceea, avem nevoie sa cerem sistemului de operare sa execute niste operatii pentru programul nostru. Aceasta delegare a muncii catre sistemul de operare se realizeaza prin intreruperi.

In continuare, discutam toate aceste componente detaliat.

3.1 Registrii arhitecturii x86

Registrii sistemului x86 pot fi sumarizati in urmatoarea diagrama:



Arhitectura procesorului este pe 32 de biti, astfel ca registrii utilizati sunt tot pe 32 de biti. Un registru reprezinta o cantiate mica de spatiu de stocare pe unitatea centrala de procesare, al carui continut poate fi accesat mai rapid decat datele stocate in alt loc (de exemplu, in memorie sau chiar mai lent pe hard-disk).

Registrii arhitecturii x86 sunt:

- cei 8 din imaginea de mai sus, numiti registrii de uz general (*General-Purpose Registers*);
- un registru pentru *flags*;
- un registru pointer la instructiunea urmatoare ce va fi executata (*Instruction Pointer*);
- alti registri.

3.1.1 Registrii de uz general

Registrul	Nume	Descriere	Restaurat dupa apel
AX	accumulator	este utilizat in operatii aritmetice	nu
BX	base	utilizat ca pointer la date	da
CX	counter	este utilizat in instructiunile repetitive	nu
DX	data	este utilizat in operatii aritmetice si I/O	nu
SP	stack pointer	pointer la varful stivei	da
BP	stack base pointer	pointer la baza stivei	da
SI	source index	pointer la sursa in operatii stream	da
DI	destination index	pointer la destinatie in operatii stream	da

Observatie. Registrul **BP** poate fi utilizat si ca *frame pointer*.

Alta observatie. Urmarim tabelul si observam ca registrii pe arhitectura x86 pe 32 de biti au prefixul **E**. In acest caz, registrul **EAX** este format din **AX** (*accumulator*), **AH** (H - *high*) si **AL** (L - *low*). De exemplu, daca in **EAX** stocam valorea 0, dar punem in **AH** ulterior valoarea 1, atunci intreg registrul **EAX** va stoca valoarea 256 in baza 10. **De ce?**

3.1.2 EFLAGS

Registrii EFLAGS (indicatori) sunt:

- **CF** - *carry flag*, care se seteaza daca dupa ultima operatie a avut loc transport;
- **PF** - *parity flag*, care este setat daca numarul de biti de 1 din rezultatul ultimei operatii este par;
- **ZF** - *zero flag*, care este setat daca rezultatul ultimei operatii a fost 0;
- **SF** - *sign flag*, care este setat daca rezultatul ultimei operatii a fost negativ;
- **OF** - *overlow flag*, care este setat daca rezultatul ultimei operatii a produs overflow.

3.1.3 Adrese

Pentru a reprezenta adresele de memorie, putem utiliza una dintre urmatoarele variante:

- valoarea unui registru pe 32 de biti: (%eax);
- suma dintre constanta numerica si valoarea unui registru: 4(%eax) (inteles ca %eax + 4);
- suma dintre doi registri: (%eax, %edx) (inteles ca %eax + %edx);
- suma dintre doi registri si o constanta numerica: 4(%eax, %edx) (inteles ca %eax + %edx + 4);
- suma a doi registri, dintre care unul inmultit cu 2, 4 sau 8, la care se poate aduna o constanta: 16(%eax, %edx, 4) (inteles ca %eax + 4 * %edx + 16);

3.1.4 Notatie

Este important sa precizam faptul ca, in scrierea programelor, registrii sunt prefixati de simbolul %. Astfel, pentru utilizarea registrului **EAX**, vom scrie *%eax* (vom pastra conventia de notatie cu litere mici).

La fel de important: asa cum registrii sunt precedati de simbolul %, constantele numerice sunt precedate de simbolul $. Vom scrie, astfel, *$4* pentru a reprezenta constanta 4.

3.2 Tipurile de date si structura de baza a unui program

3.2.1 Tipurile de date

Tipurile principale de date pe care le vom utiliza sunt

1. **byte** - dupa cum ii spune si numele, ocupa 1 byte, adica 8 biti in memorie;
2. **single** - ocupa 4 bytes (32 de biti) si este utilizat pentru stocarea numerelor fractionare;
3. **word** - ocupa 2 bytes (16 biti) si este utilizat pentru stocarea intregilor;
4. **long** - ocupa 4 bytes (32 de biti) si este utilizat pentru a stoca intregi pe 32 de biti sau numere fractionare pe 64 de biti. **Double word** (dword) ocupa tot 32 de biti;
5. **quad** - ocupa 8 bytes (64 de biti);
6. **ascii** - este utilizat pentru declararea sirurilor de caractere care nu sunt finalizate cu terminatorul de sir - nerecomandat;
7. **asciz** - este utilizat pentru declararea sirurilor de caractere care sunt finalizate cu terminatorul de sir;
8. **space** - defineste un spatiu in memorie, a carui dimensiune o specificam, de exemplu **.space 4**, insemnand ca se lasa 4 bytes = 32 de biti. Este util atunci cand declaram variabile pe care le calculam in cadrul programului si pentru care nu vrem initial o valoare default.

Pentru a declara o variabila, sintaxa va fi

```
nume: .tip valoare
```

de exemplu

```
x: .word 15
y: .floating 23.74
str: .asciz "String"
```

3.2.2 Structura unui program

Structura de baza a unui program Assembly x86 este:

```
.data
    ;// aici declaram variabilele programului

.text
    ;// de aici incepe codul propriu-zis
    ;// definim main ca fiind o eticheta globala
    ;// reprezentand entry-ul din programul nostru

    ;// in principiu, in acest loc vom scrie procedurile
    ;// aici este scris programul principal
    ;// echivalentul main-ului
.global main
main:
    ;// instructiuni
```

3.3 Exemplu de program in x86

Important. Instructiunile in limbajul de asamblare au forma

```
[eticheta] mnemonic/operatie [operanzi]
```

3.3.1 Instructiunea MOV

Este o instructiune avand sintaxa

```
mov source, destination
```

cu efectul de a muta sursa in destinatie (atribuire din stanga in dreapta, *source* → *destination*). O informatie interesanta referitoare la aceasta instructiune este faptul ca **mov** e *Turing-complete*[12] (vezi si problema 11 de la Exercitii propuse).

Sursa si destinatia pot fi:

- mov registru, registru; (*mov %eax, %ebx*, i.e., %ebx ← %eax)
- mov adresa de memorie, registru;
- mov registru, adresa de memorie;
- mov constanta numerica, registru (*mov $16, %edx*, i.e., %edx ← 16);

Important. Instructiunea MOV poate fi sufixata de dimensiunea tipului de data, astfel ca putem avea **movl** (pentru long), **movq** (pentru quad) etc. Instructiunile cu sufix permit si o varianta de instructiune in care sursa este o constanta, iar destinatia o adresa de memorie.

3.3.2 Intreruperi de sistem

Pentru a putea scrie un prim program in Assembly, trebuie sa ne asiguram ca ii putem opri executia. Daca in limbajele *medium* si *high-level* de acest lucru se ocupa compilatorul, acum sarcina ii revine programatorului. Prima intrerupere pe care o discutam este **SYSTEM EXIT**.

Pentru a apela o intrerupere, vom folosi instructiunea **int**. Cea mai comuna (cea pe care o vom utiliza cel mai des) este

```
int $0x80
```

care inseamna **system call** (0x80 este hardcodat si este asociat intreruperii care face un apel sistem). System call inseamna ca cerem sistemului de operare sa faca ceva pentru noi (ceva ce programul nostru nu are "drepturi" sa faca direct).

Pentru ca sistemul sa stie de ce este apelat, trebuie sa specificam inca doua lucruri:

- ce functie apelam;
- ce parametri are functia respectiva.

In general, programele C/C++ le finalizam cu un `return 0;`, care este echivalent cu apelul sistem `exit(0)`. Identificam, deci:

[12] Computerphile, `https://www.youtube.com/watch?v=RPQD7-AOjMI`

- functia apelata este **exit**;
- parametrul (argumentul) ei este **0**.

Asa cum si apelul sistemului (system call) este identificat in mod unic prin codul 0x80, si functiile sistem sunt identificate in mod unic prin numere. In acest caz, functiei **exit** ii corespunde numarul 1, iar **registrul** care retine ce functie este apelata este **%eax**. Argumentele sunt date prin urmatorii registri: **%ebx**, **%ecx** etc. (puteti consulta tabelul acesta pentru a vedea parametrii functiilor [13]

Punand aceste informatii cap la cap, intreruperea sistem pentru oprirea programului este:

```
mov $1, %eax
mov $0, %ebx
int $0x80
```

Recapituland:

- cand sistemul depisteaza intreruperea 0x80, intelege ca are un **system call** si isi verifica registrii;
- registrul **%eax** retine codul functiei pe care trebuie sa o apeleze, in acest caz codul **1** inseamna functia **exit**. De unde stim acest cod? Cautam in documentatia sistemului de operare[14];
- stiind ca apeleaza exit care are un argument, se uita in %ebx pentru a citi valoarea acestuia.

Observatie: exista diferite tipuri de apeluri sistem, pe care le vom explica la momentul potrivit. De exemplu, afisarea pe ecran / in fisier text este un apel sistem, citirea de la tastatura / din fisier text este un apel sistem etc.

Observatie: Ce se intampla daca avem un apel de sistem cu multi parametrii? Amintiti-va ca avem un numar mic de registrii. Lucrurile se complica putin daca avem mai mult de 5 parametrii[15].

3.3.3 Un prim program: cod sursa

Avand toate informatiile anterioare, vom scrie primul program complet in limbaj de asamblare:

```
.data
.text
.global main
main:
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Pentru a putea rula programul vom executa urmatoarele comenzi in terminal:

```
gcc --m32 program_exit.asm -o program_exit

./program_exit
```

[13] https://chromium.googlesource.com/chromiumos/docs/+/master/constants/syscalls.md#x86-32_bit
[14] https://github.com/opennetworklinux/linux-3.8.13/blob/master/arch/sh/include/uapi/asm/unistd_32.h
[15] http://asm.sourceforge.net/articles/linasm.html#Syscalls

Observatie: Pentru a functiona varianta pe 32 de biți ținând cont că sistemul de operare este pe 64 de biți, va trebui să instalați o bibliotecă suplimentară folosind comanda **sudo apt-get install g++-multilib**.

Observatie: Daca in timpul rularii comenzii gcc de mai sus intampinati eroarea:

```
/usr/bin/ld:program.asm: file format not recognized; treating as linker script

/usr/bin/ld:program.asm: syntax error

collect2: error: ls returned 1 exit status
```

Atunci trebuie sa redenumiti fisierul in program_exit.S (nu .asm, ci .S) si sa incercati din nou comanda gcc.

Observatie: Este posibil ca dupa compilare sa vedeti un avertisment de tipul:

```
/usr/bin/ld: warning: relocation in read-only section '.text'

/usr/bin/ld: warning: creating DT_TEXTREL in a PIE
```

Acest avertisment este generat de un mecanism de protectie a memoriei care se numeste PIE (Position Independent Executables). Vom discuta despre ce inseamna acest lucru mai tarziu pe parcursul materiei.

Pentru a nu mai obtine aceste avertismente, este suficient sa adaugati un flag `-no-pie` in comanda, astfel incat sa aiba urmatoarea forma:

```
gcc --m32 program_exit.s -o program_exit -no-pie

./program_exit
```

3.4 Programe in ASM si Debug

Pentru a putea intelege mai bine cum isi modifica registrii valorile, dar si pentru a putea corecta anumite *bug*-uri cand apar in codul scris, este important sa stim sa utilizam un *debugger*. *Debugging* este o activitate cu care se confrunta orice programator, indiferent de limbajul de programare folosit. Pentru inceput, fie urmatorul program scris in limbaj de asamblare (sa denumim fisierul **program2.asm**):

```
.data
    x: .long 15
.text
.global main

main:
    movl $0, %eax
    movl %eax, x
    movl x, %ebx

    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Executam

```
gcc -m32 program2.asm -o program2
```

Pentru a putea face debug, vom utiliza **gdb**. Rulam in terminal

```
gdb program2
```

Punem un breakpoint la adresa de intrare in program, si anume la **main**:

```
b main
```

Rulam programul

```
run
```

Iar acum, vom executa succesiv

```
i r
stepi
```

(i. e. next step si information registers) pentru a face debug *step by step*. Urmariti cum se modifica valoarea pentru registrii %eax si %ebx.

3.4.1 Un alt program

```
.data
.text
.global main
main:
```

```
movl $0, %eax
mov $1, %ah

mov $1, %eax
mov $0, %ebx
int $0x80
```

Ce este stocat in registrul %eax dupa executarea instructiunii `mov $1, %ah`? De ce? Care ar fi fost valoarea lui %eax daca am fi avut %al in loc de %ah?

3.5 Hello, world! in Assembly

In aceasta sectiune vom face o intrerupere de sistem pentru afisarea unui sir de caractere pe ecran.

La fel ca pentru **SYSTEM EXIT**, si **PRINT STRING** va avea un cod de functie (in %eax) si un set de argumente (in acest caz sunt 3: %ebx, %ecx si %edx).

Astfel, pentru a putea afisa un string pe ecran, urmarim pasii:

- incarcam in registrul %eax valoarea 4, corespunzatoare functiei *write* (vezi link-ul de mai sus unde sunt listate apelurile de sistem si codurile asociate);
- incarcam in registrul %ebx valoarea 1, corespunzatoare locului in care afisam textul - in consola, adica la **standard output**;
- incarcam in %ecx mesajul de afisat, in acest caz un sir din memorie;
- incarcam in %edx lungimea sirului afisat. Atentie la declararea cu .asciz, pentru ca se considera la lungime si terminatorul de sir.

Fie urmatorul program Assembly complet:

```
.data
    helloWorld: .asciz "Hello, world!\n"
.text
.global main
main:
    mov $4, %eax
    mov $1, %ebx
    mov $helloWorld, %ecx
    mov $15, %edx
    int $0x80

    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Nota: Testati-va cunostintele acumulate pana acum facand testele de laborator 2.1, 2.2 si 2.3 pe care le gasiti online la sectiunea laboratoare.

3.6 Operatii aritmetice si logice

3.6.1 Operatii aritmetice

In aceasta parte a laboratorului vom prezenta cele patru operatii aritmetice de baza - adunarea, scaderea, inmultirea si impartirea prin cat si rest.

Instructiune	Efect
add op1, op2	op2 := op2 + op1
sub op1, op2	op2 := op2 - op1
mul op	(edx, eax) := eax × op
imul op	(edx, eax) := eax × op
div op	(edx, eax) := (edx, eax) / op
idiv op	(edx, eax) := (edx, eax) / op

Observatie: pentru `div`, scrierea `(edx, eax)` inseamna ca se imparte $2^{32} * edx + eax$ la `op`. Daca nu vrem sa impartim un numar foarte mare, trebuie sa punem in `edx` valoarea 0 initial.

Explicatii inmultire si impartire.

- atat inmultirea, cat si impartirea au operanzi impliciti, acestia fiind EDX si EAX. Pentru a inmulti un numar cu o anumita valoare (sau a imparti la un numar o anumita valoare), trebuie sa avem deja informatia completata in registrul EAX;
- trebuie sa diferentiem intre MUL si IMUL, intre DIV si IDIV, instructiunea precedata de I fiind pentru prelucrarea numerelor **cu semn**;
- pentru inmultire, rezultatul este dat de $2^{32} \times edx + eax$;
- pentru impartire, eax stocheaza catul, iar edx stocheaza restul;
- in cazul inmultirilor si al impartirilor, operandul *op* **NU** poate fi o constanta numerica! In rest, *op* poate fi registru sau locatie de memorie.

3.6.2 Exemplu

Sa se scrie un program care sa afiseze suma, diferenta, produsul, precum si catul si restul impartirii a doua numere intregi cu semn.

Solutie.

```
.data
    x: .long 30
    y: .long 7
    sum: .space 4
    dif: .space 4
    prod: .space 4
    cat: .space 4
    rest: .space 4
.text
.global main
```

```
main:
    mov x, %eax
    add y, %eax
    mov %eax, sum

    mov x, %eax
    sub y, %eax
    mov %eax, dif

    mov x, %eax
    mov y, %ebx
    imul %ebx
    mov %eax, prod

    mov x, %eax
    mov y, %ebx
    mov $0, %edx
    idiv %ebx
    mov %eax, cat
    mov %edx, rest

    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Important. Pentru a inspecta valoarea stocata in variabilele declarate cu **.space 4**, putem folosi tot debugger-ul **gdb**, executand la un pas:

```
print (long) nume_variabila
```

de exemplu,

```
stepi
print (long) sum
```

3.6.3 Operatii logice

Operatiile logice sunt **and**, **or**, **xor** si **not**, primele trei avand structura `operatie sursa, destinatie`, iar **not** avand structura `not destinatie`.

Instructiune	Efect
not op	op := ~op
and op1, op2	op2 := op2 & op1
or op1, op2	op2 := op2 | op1
xor op1, op2	op2 := op2 ^ op1

Cerinta. Scrieti un program pentru a testa cele patru operatii de mai sus. In loc de **and**, folositi si operatia **test**, si comparati, utilizand debuggerul **gdb**, efectul dintre aceasta si **and**.

3.6.4 Operatii de shift (deplasare) logice

Operatiile logice de deplasare sunt **shr**, **shl**, **sar** si **sal**, toate avand structura `operatie numar, destinatie` unde *numar* este cati biti sunt deplasati. In numele instructiunii, s este *shift*, distinctia l/r este pentru left/right (stanga/dreapta) iar distinctia h/a este pentru logic/arithmetic (deplasare pe biti sau aritmetica). In cazul deplasarilor aritmetice, bitul cel mai semnificativ isi pastreaza valoarea (se pastreaza semnul numarului).

Instructiune	**Efect**
shr numar, op	op := op $\gg$ numar
shl numar, op	op := op $\ll$ numar
sar numar, op	op2 := op $\gg$ numar (cu pastrare semn op)
sal numar, op	op2 := op $\ll$ numar (cu pastrare semn op)

Cerinta. Scrieti un program pentru a testa cele patru operatii de mai sus.

3.7 Salturi conditionate si neconditionate

3.7.1 Salturile neconditionate

In foarte multe cazuri avem nevoie sa putem sari la o anumita instructiune in cod (ganditi-va la if-uri si cicluri repetitive). In aceasta sectiune discutam modurile in care salturile in cod se pot realiza.

Se executa prin instructiunea

```
jmp et
```

sarind la eticheta **et** a programului (se efectueaza un salt in memorie, fara a se tine cont de **IP**).

De exemplu:

```
.data
    text1: .asciz "Text 1\n"
    text2: .asciz "Text 2\n"
.text
.global main

main:
    jmp et2
    mov $4, %eax
    mov $1, %ebx
    mov $text1, %ecx
    mov $8, %edx
    int $0x80

et2:
    mov $4, %eax
    mov $1, %ebx
    mov $text2, %ecx
    mov $8, %edx
    int $0x80

    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

In acest caz, valoarea afisata pe ecran este string-ul “Text 2”.

3.7.2 Salturi conditionate

Instructiunea este

```
j<conditie> et
```

si se executa doar daca este respectata conditia ceruta, sarind la eticheta **et**. In caz contrar, se executa linia urmatoare de cod.

Pot fi testate atat *flag*-urile deja prezentate - carry, overflow, zero, sign, cat pot fi executate si instructiuni corespunzatoare operatorilor relationali ($<$, $<=$, $>$, $>=$, $!=$, $==$).

Operator	Descriere
jc	jump daca este carry setat
jnc	jump daca nu este carry setat
jo	jump daca este overflow setat
jno	jump daca nu este overlow setat
jz	jump daca este zero setat
jnz	jump daca nu este zero setat
js	jump daca este sign setat
jns	jump daca nu este sign setat
Operatori pentru numere fara semn	
jb	jump if below (op1 < op2)
jbe	jump if below or equal (op1 <= op2)
ja	jump if above (op1 > op2)
jae	jump if above or equal (op1 >= op2)
Operatori pentru numere cu semn	
jl	jump if less than (op1 < op2)
jle	jump if less than or equal (op1 <= op2)
jg	jump if greater than (op1 > op2)
jge	jump if greater than or equal (op1 >= op2)
Operatori de egaliate	
je	jump if equal (op1 == op2)
jne	jump if not equal (op1 ! = op2)

Observatie. Pentru a putea utiliza unul dintre operatorii de mai sus, trebuie sa putem compara valori. In acest caz, utilizam instructiunea **cmp**.

Atentie! Daca vom compara **cmp %eax, %ebx**, atunci operatia de $\leq$ trebuie privita ca $\%ebx \leq \%eax$.

```
.data
    a: .long 5
    b: .long -3
    text1: .asciz "a >= b\n"
    text2: .asciz "a < b\n"
.text

.global main

main:
    mov a, %eax
    mov b, %ebx
    cmp %ebx, %eax
    jge et

    mov $4, %eax
    mov $1, %ebx
    mov $text2, %ecx
    mov $7, %edx
```

```
    int $0x80

    jmp exit

et:
    mov $4, %eax
    mov $1, %ebx
    mov $text1, %ecx
    mov $8, %edx
    int $0x80

exit:
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Cerinta. Rulati programul de mai sus si executati-l cu debuggerul, pentru a vedea cum se modifica valorile.

3.7.3 Simularea structurilor repetitive

Sa presupunem ca vrem sa simulam urmatorul program dezvoltat in C:

```c
#include <stdio.h>

int main()
{
    int n = 5, s = 0;
    for (int i = 0; i < n; i++)
    {
        s += i;
    }
    printf("%d\n", s);
    return 0;
}
```

3.7.4 Simularea structurilor repetitive: solutie 1

In programul Assembly urmator vom vedea cum "simulam" cicluri de calcul cu instructiuni de salt si etichete definite corespunzator.

```
.data
    n: .long 5
    s: .space 4
.text

.global main

main:
```

```
    mov $0, %ecx
    ;// ecx era registrul-contor

etloop:
    cmp n, %ecx
    je etexit
    ;// daca %ecx == n am terminat parcurgerea

    add %ecx, s
    ;// s += %ecx

    add $1, %ecx
    ;// %ecx += 1

    jmp etloop;
    ;// reiau parcurgerea

etexit:
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Putem folosi debuggerul pentru a vedea rezultatul. Sa presupunem ca programul se numeste **suma.asm** si am ajuns sa il rulam cu **./suma**. Putem folosi **gdb**-ul astfel:

```
gdb suma
b etexit
run
print (int) s
```

3.7.5 Simularea structurilor repetitive: solutie 2

O solutie mai apropiata de particularitatile de limbaj este cea care utilizeaza *keyword*-ul **loop**. Actiunea acestuia este simpla, se specifica `loop eticheta`, insemnand ca se itereaza in cadrul etichetei date (in programul anterior, aceea era **etloop**), si se opreste executarea cand registrul %ecx devine 0. Atentie la faptul ca %ecx este decrementat automat, nu trebuie sa facem manual aceasta modificare. Rescriem, asadar, programul anterior:

```
.data
    n: .long 5
    s: .space 4
.text

.global main

main:
    sub $1, n
    mov n, %ecx
```

```
etloop:
    add %ecx, s
    loop etloop
    jmp etexit

etexit:
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Cerinte:

1. Explicati necesitatea instructiunii `sub $1, n` (verificati documentatia x86 pentru instructiunile *inc* si *dec*);
2. Rulati programul in **gdb** pentru a va asigura de corectitudinea lui.

Nota: Testati-va cunostintele acumulate pana acum facand testele de laborator 3.1 si 3.2 pe care le gasiti online la sectiunea laboratoare.

3.8 Tablouri unidimensionale

Pentru a putea discuta despre tablourile unidimensionale (de intregi, momentan), trebuie sa intelegem adresarea in memorie si sa invatam o noua operatie - Load Effective Address (LEA).

3.8.1 Adresarea in memorie

Fie urmatoarea scriere

```
a(b, c, d)
```

pe care o vom intelege ca fiind $b+c\times d+a$. Mai exact, daca o vom transpune in registri si constante, -8(%ebx, %eax, 4) inseamna $\%ebx + \%eax \times 4 - 8$, iar scrierea

```
mov $5, -8(%ebx, %eax, 4)
```

inseamna ca salvam constanta 5 la adresa de memorie calculata ca fiind $\%ebx + \%eax \times 4 - 8$.

Uneori, anumite informatii pot sa lipseasca, si atunci sa avem, de exemplu, instructiunea

```
mov $0, (%edi, %ecx, 4)
```

Atentie! Cand lucram cu *arrays*, nu putem lasa doar **mov**, ci va trebui sa sufixam cu tipul de date necesar (de exemplu **movl** pentru *arrays* cu elemente de tip **long**).

3.8.2 Load Effective Address (LEA)

Instructiunea **lea** incarca adresa sursei in destinatie, si are forma

```
lea source, destination
```

De exemplu,

```
lea v, %edi
```

incarca in registrul %edi adresa lui v.

3.8.3 Reprezentarea in memorie a tablourilor unidimensionale

Tablourile unidimensionale sunt liniar stocate in memorie, plecand de la o adresa initiala si ocupand, succesiv, atatea locatii de memorie cat ocupa si tipul de date utilizat.

De exemplu, daca tipul de date este **long**, atunci fiecare element al tabloului ocupa o locatie egala cu 4 (long este pe 4 bytes). In acest caz, elementele se vor gasi, relativ la locatia initiala de memorie, astfel:

- primul element, la locatia de memorie initiala, MEM, la care se adauga dimensiunea tipului de date inmultita cu numarul de ordine al elementului, adica MEM + 4 × 0;
- al doilea element se afla la locatia de memorie MEM, la care se adauga produsul dintre 4 (dimensiunea tipului de date) × 1 (indexul curent): MEM + 4 × 1;
- al treilea element se afla la MEM + 4 × 2;
- etc.

Adresa de memorie o vom stoca in registrul %edi (destination index register), iar pentru contorul indexului curent vom utiliza registrul %ecx. In acest caz, observam ca, utilizand scrierea anterioara, in triplet, obtinem elementele sub forma (%edi, %ecx, 4) adica $\%edi + 4 \times \%ecx$.

3.8.4 Declararea tablourilor unidimensionale

Se declara ca o eticheta in memorie, cu elementele insiruite:

```
    v: .long 10, 20, 30, 40, 50
```

si, in general, vom stoca in memorie si numarul de elemente:

```
    n: .long 5
```

3.8.5 Exemple: programul 1

Fie urmatorul program complet Assembly:

```
.data
    n: .long 5
    v: .long 10, 20, 30, 40, 50
.text
.global main

main:
    lea v, %edi
    mov $0, %ecx
    movl (%edi, %ecx, 4), %edx

etexit:
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Rulati acest program cu **gdb** si gasiti valoarea stocata in %edx la eticheta **etexit**. Modificati valoarea contorului si observati liniaritatea elementelor in memorie.

3.8.6 Exemple: programul 2

Fie urmatorul program complet Assembly:

```
.data
    n: .long 5
    v: .long 10, 20, 30, 40, 50
.text
.global main

main:
    lea v, %edi
    mov n, %ecx

etloop:
    mov n, %eax
    sub %ecx, %eax
    movl (%edi, %eax, 4), %edx
```

```
    loop etloop

etexit:
    mov $1, %eax
    mov $0, %ebx
    int $0x80
```

Cerinte.

1. Intelegeti ce face acest program, justificand necesitatea registrului %eax.
2. Rulati in **gdb** utilizand instructiunile

   ```
   gdb nume_program
   b etloop
   i r
   c
   ```

   Comanda **c** este utilizata pentru a continua debug-ul, astfel ca va sari mereu in eticheta **etloop**. Cu **i r** putem vedea continutul memoriei, pentru a analiza valorile stocate in %edx.
3. Reusiti sa gasiti valoarea 50 pastrand doar acest breakpoint? Ce ar trebui modificat pentru a putea gasi si %edx = 50?

Nota: Testati-va cunostintele acumulate pana acum facand testele de laborator 4.1, 4.2 si 4.3 pe care le gasiti online la sectiunea laboratoare.

3.9 Despre inline Assembly

Assembly x86 fi utilizat si pentru a scrie *inline assembly* direct in fisiere C. Tot ce trebuie facut este sa se adauge directiva `__asm__`, in interiorul careia vor fi scrise instructiuni de asamblare, precum in laboratorul curent. In exemplul de mai jos, plecam de la o variabila x si ii adaugam 1 valorii curente pe care o stocheaza.

Observatie. Ignorati, pentru moment, instructiunile *pusha* si *popa*; acestea vor fi explicate in laboratoarele urmatoare. Tot ca observatie, incercati sa mutati variabila x in interiorul functiei *main()* si observati efectul.

Presupunem ca avem urmatorul program, salvat in fisierul **file.c**:

```
#include <stdio.h>
int x = 1;
int main()
{
    __asm__
    (
        "pusha;"

        "mov x, %eax;"
        "add $1, %eax;"
        "mov %eax, x;"

        "popa;"
    );

    printf("%d\n", x);
    return 0;
}
```

Programul se va compila si executa cu

```
gcc file.c -o file -m32
./file
```

Cerinta. Implementati si alte programe din acest suport de laborator, utilizand **inline assembly**.

4 Exercitii propuse

4.1 Laboratorul 1

1. Se dau doua numere naturale **x** si **y**. Sa se scrie un program in asamblare care sa realizeze interschimbarea lor. Sa se observe efectul folosind debugger-ul.

2. Fie **s** un spatiu alocat de 12 octeti. Folosind codul functiei read (vezi link-ul de mai sus unde sunt listate apelurile de sistem si codurile asociate) si stiind ca regula de incarcare este aceeasi ca in cazul functiei write, cititi in **s** de la tastatura (codul pentru stdin este 0) 12 caractere. Afisati-l pe **s**.

4.2 Laboratorul 2

1. Fie **x**, **y** doua numere natruale salvate in memorie. Sa se scrie un program care calculeaza in doua moduri (x/16) + (y $\times$ 16).

2. Sa se adauge in cadrul programului anterior la final un pas pentru a verifica daca cele doua valori calculate sunt egale. Daca da, sa se afiseze "PASS", altfel "FAIL".

3. Fie **a, b, c** trei numere salvate in memorie si **min** un spatiu alocat de 4 octeti. Sa se salveze in **min** cel mai mic numar dintre cele trei.

4. Sa se verifice daca un numar dat este prim (folositi ambele variante de structuri repetitive).

4.3 Laboratorul 3

1. Sa se determine maximul si numarul de aparitii al acestuia intr-un array.

2. Fie **s** un string salvat in memorie si **t** un spatiu alocat cu aceeasi numar de octeti. Sa se obtina in **t** inversul string-ului **s** si sa se afiseze pe ecran.

5 Exercitii suplimentare

1. Scrieti secvente de cod Assembly x86 care calculeaza urmatoarele:
   (a) EAX $\leftarrow$ EAX & 1
   (b) EAX $\leftarrow$ EAX ˆ EAX
   (c) EAX $\leftarrow$ (EAX ˆ EBX) ˆ EBX
   (d) EAX $\leftarrow$ EAX & (EAX $-$ 1)
   (e) EAX $\leftarrow$ (EAX & $\sim$EBX) | ($\sim$EAX & EBX)
   (f) EAX $\leftarrow$ (EAX & $\sim$EAX) | ($\sim$EBX & EBX)
   (g) EAX $\leftarrow$ (EAX | $\sim$EBX) & ($\sim$EAX | EBX)
   (h) EAX $\leftarrow$ (EAX | $\sim$EAX) & ($\sim$EBX | EBX)
   (i) EAX $\leftarrow$ EBX ˆ ((EAX ˆ EBX) & $-$(EAX $<$ EBX))
   (j) EAX $\leftarrow$ EAX ˆ ((EAX ˆ EBX) & $-$(EAX $<$ EBX))
   (k) ECX $\leftarrow$ EAX + EBX - ECX
   (l) EBX $\leftarrow$ ((EAX $\times$ 2 + EBX) $\times$ 2 + ECX)/2
   (m) EAX $\leftarrow$ (EAX + EBX + ECX) $\times$ 16
   (n) ECX $\leftarrow$ (EAX/16) + (EBX $\times$ 16)
   (o) ECX $\leftarrow$ (EAX & 0xFFFF0000) + (EBX & 0x0000FFFF)
   (p) EAX $\leftarrow$ (EAX + EBX)/(EAX + ECX)
   (q) EAX $\leftarrow$ (EAX$\times$EBX)/(ECX^2)
   (r) EAX $\leftarrow$ EAX$\times$((EAX+EBX)/ECX+1)
   (s) EAX $\leftarrow$ 1 + EAX + EAX^2/2 + EAX^3/6 + EAX^4/24 + ...

2. In registrul EAX avem un numar intreg. Verificati daca numarul este negativ, pozitiv, sau zero.

3. In registrii EAX si EBX avem doua valori distincte. Implementati SWAP(EAX, EBX) folosind doar functii logice si niciun registru suplimentar sau memorie. (verificati daca x86/x87 are deja o instructiune care realizeaza aceasta operatie, daca da atunci ganditi-va de ce este importanta aceasta operatie)

4. In registrul EAX aveti un numar natural. Gasiti cea mai apropiata putere de 2 mai mare decat acest numar. (verificati daca x86/x87 are deja o instructiune care realizeaza aceasta operatie, daca da atunci ganditi-va de ce este importanta aceasta operatie)

5. Se da un numar natural in registrul EAX, numarati cati biti de "1" sunt in reprezentarea binara a numarului. (verificati daca x86/x87 are deja o instructiune care realizeaza aceasta operatie, daca da atunci ganditi-va de ce este importanta aceasta operatie)

6. Se da un numar natural strict pozitiv in registrul EAX, calculati $\lfloor \log_2 \text{EAX} \rfloor$. (verificati daca x86/x87 are deja o instructiune care realizeaza aceasta operatie, daca da atunci ganditi-va de ce este importanta aceasta operatie)

7. Se da un numar natural in registrul EAX, calculati dimensiunea celui mai lung sir de biti "1" consecutivi in reprezentarea binara a numarului.

8. Se da un numar natural in registrul EAX, verificati daca numarul este un palindrom (in reprezentarea binara).

9. Se da un numar intreg cu semn, sa se determine toti divizorii acestuia.

10. Utilizand doar instructiuni **mov**, sa se obtina intr-un registru numarul 2318. (fara a utiliza instructiunea `mov $2318, %eax`)

11. Sa se determine cel de-al n-lea termen al sirului Fibonacci, unde n este dat de la tastatura. Nu se garanteaza ca este un numar intreg pozitiv.

12. Sa se calculeze suma dintr-un tablou unidimensional. Sa se calculeze si media aritmetica (impartire prin cat si rest) a elementelor din tablou.

13. Calculati EAX $\leftarrow$ EAX! (EAX factorial).

14. Sa se determine cele mai mici doua elemente dintr-un array, utilizand o singura parcurgere.

15. Fie dat un array in memorie de lungime impara. Stim ca toate elementele sunt duplicate, cu exceptia unuia, de exemplu `v = { 1245, 342, 543523, 342, 4234, 1245, 543523 }`, unde elementul `4234` este elementul singular. Sa se scrie un algoritm eficient care sa determine acest element (complexitate spatiu $O(1)$ si complexitate timp $O(n)$, unde n este lungimea array-ului).

16. Utilizand algoritmul de cautare binara, sa se verifice daca un array contine un anumit element. Daca da, se va determina pozitia acelui element, altfel pozitia va fi -1.

19*. Sa se calculeze radicalul intreg al unui numar (integer square root)[16]. (pentru un calcul aproximativ rapid al radicalului vedeti o secventa de cod ajunsa celebra: codul sursa pentru Q_rsqrt din Quake[17])

20*. Sa se sorteze un tablou unidimensional. Sa se scrie un program C care foloseste cat mai multe instructiuni scrise inline assembly pentru a sorta un tablou unidimensional (ASM pur si inline ASM).

21*. Pe urmele primului programator, Ada Lovelace, sa se determine cel de-al n-lea termen al sirului Bernoulli[18].

[16] `https://en.wikipedia.org/wiki/Integer_square_root`

[17] Q_rsqrt, `https://github.com/id-Software/Quake-III-Arena/blob/dbe4ddb10315479fc00086f08e25d968b4b43c49/code/game/q_math.c`

[18] `https://writings.stephenwolfram.com/2015/12/untangling-the-tale-of-ada-lovelace/`

6 Resurse suplimentare

Tutoriale si explicatii suplimentare se gasesc la urmatoarele resurse online.

Pentru sintaxa AT&T:

1. Adam Ferrari, x86 Assembly Guide[19];
2. Alexander Mishurov, AT&T assembly syntax and IA-32 instructions[20].

Pentru sintaxa Intel:

1. Vlad Radulescu, Laboratoare ASM[21];
2. David Evans, x86 Assembly Guide[22];
3. Cristian Botau, Introducere in asamblare[23].

[19] http://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html
[20] https://gist.github.com/mishurov/6bcf04df329973c15044
[21] https://sites.google.com/view/danton-asm/asm1?authuser=0
[22] http://www.cs.virginia.edu/~evans/cs216/guides/x86.html
[23] https://www.infoarena.ro/introducere-in-asamblare

Arhitectura Sistemelor de Calcul
Laborator - Partea 0x01

Bogdan Macovei
Ruxandra Bălucea
Cristian Rusu

Noiembrie 2022

Cuprins

1 Stiva **2**

2 Proceduri in limbaj de asamblare **4**
2.1 Apelul unei proceduri 4
2.2 Argumentele unei proceduri 5
2.3 Crearea cadrului de apel 7
2.4 Registrii callee si caller saved 9
2.5 Returnarea valorilor 11

3 Apelul functiilor din C **12**
3.1 Apelul functiei `printf` 12
3.2 Apelul functiei `scanf` 14
3.3 Un exemplu de citire si de afisare a unui graf neorientat 15

4 Manipularea numerelor în viruglă mobilă în x86 AT&T **19**

5 Exerciții **22**

6 Resurse suplimentare **24**

1 Stiva

Stiva este o regiune dinamică în cadrul unui proces care funtioneaza pe principul LIFO (Last In – First Out). La fiecare apel de funcție, este alocată o zonă de memorie pe stivă în care vor fi stocate **argumentele funcției, variabilele locale** și **adresa de retur**. La fiecare alocare stiva va crește "în jos", spre adrese mici, așa cum este prezentat și în figura 1.

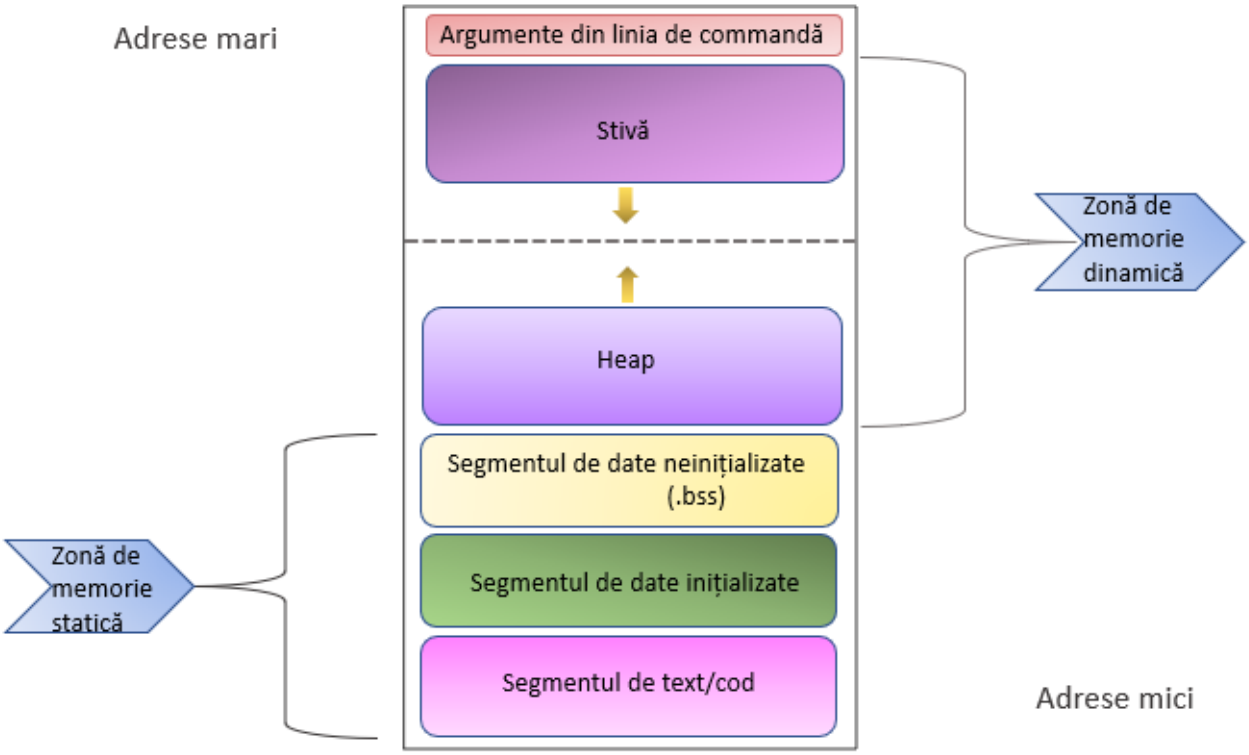


Figura 1: Spațiul de adresă al unui proces

Pentru operarea cu stiva, exista doua instructuni in x86:

```
push operand
```

unde **operand** poate fi o constanta, un registru sau o adresa de memorie. Este folosita pentru adaugarea pe stiva;

```
pop operand
```

unde **operand** poate fi un registru sau o adresa de memorie. Este folosita pentru scoaterea de valori de pe stiva.

Observatie: Pot fi adaugate pe stiva doar **word**-uri și **long**-uri.

Stack pointer-ul %esp este registrul care indică întotdeauna spre vârful stivei și prin care se efectueaza, de fapt, push-urile si pop-urile. De exemplu, la fiecare **pushl** efectuat, %esp scade cu 4 (stiva crește spre adrese mici) si este salvata o valoare relativ la adresa de memorie indicata de %esp. Pentru fiecare **popl** efectuat, valoarea din varful stivei este depozitata in operandul instructiunii, iar %esp creste cu 4.

Urmarind scrierea studiata in laboratorul trecut a(b, c, d) , valoarea efectiva din varful stivei este data de **0(%esp)**. In plus, trebuie remarcat ca varful stivei va indica intotdeauna spre ultimul byte al ultimului element adăugat pe stiva.

Base pointer-ul este registrul care indică întotdeauna spre baza stivei și prin care se va face delimitarea cadrelor de apel și accesarea elementelor corespunzătoare acestora.

In continuare este prezentat un exemplu pentru evaluarea expresiei

$$((x + y) * (x - y) * (x + z))/(y + z)$$

folosind stive. x, y, z sunt 3 numere de tip long declarate global.

```
 .data
x: .long 2
y: .long 1
z: .long 3
e: .space 4
.text
.global main
main:

movl y, %eax
addl z, %eax
pushl %eax
; %esp: (y + z)

movl x, %eax
addl z, %eax
pushl %eax
; %esp: (x + z), (y + z)

movl x, %eax
subl y, %eax
pushl %eax
; %esp: (x - y), (x + z), (y + z)

movl x, %eax
addl y, %eax
pushl %eax
; %esp: (x + y), (x - y), (x + z), (y + z)
```

```
popl %eax
popl %ebx
mull %ebx

pushl %eax
; %esp: (x + y) * (x - y), (x + z), (y + z)

popl %eax
popl %ebx
mull %ebx

pushl %eax
; %esp: (x + y) * (x - y) * (x + z), (y + z)

popl %eax
popl %ebx
divl %ebx

pushl %eax
; %esp: ((x + y) * (x - y) * (x + z)) / (y + z)

popl e

mov $1, %eax
xor %ebx, %ebx
int $0x80
```

Observatie: Atentie la comentariile din codul de mai sus! Ca si pana acum, in functie de ce environment utilizati, comentariile pot fi introduse ori cu `#`, ori cu `//`;

2 Proceduri in limbaj de asamblare

Daca o prelucrare trebuie facuta de mai multe ori in acelasi fel (eventual cu alte date), in loc sa rescriem grupul respectiv de instructiuni de mai multe ori in program putem sa-l incapsulam intr-o procedura si de fiecare data cand avem nevoie de el sa apelam procedura (eventual cu noile date transmise ca parametri).

In x86 procedurile nu au un mod specific de definire. Ele sunt simple blocuri de cod ape- late insa intr-o maniera specifica. Evident, ele trebuie declarate in zona **.text** si sunt identificate printr-un nume sub forma unei etichete.

2.1 Apelul unei proceduri

Mecanismul de control privind salturile intr-o procedura si revenirea in locul de unde a fost apelata este realizat prin doua instructiuni - **call** si **ret**.

```
call proc
```

unde **proc** este eticheta ce marcheaza inceputul procedurii. Prin aceasta instructiune se apeleaza procedura **proc** si se retine adresa de intoarcere (locul de unde procedura a fost apelata). De fapt, se produce o salvare pe stiva a valorii din %eip (instruction pointer) care retine fix adresa instructiunii urmatoare. In acelasi timp, %eip preia adresa primei instructiuni din procedura, realizandu-se astfel continuarea executiei din acel punct.

```
ret
```

Aceasta instructiune realizeaza un salt la adresa din varful stivei. Cum pe parcursul procedurii apelate toate elementele adaugate pe stiva sunt si eliminate, la final in varful stivei se va afla chiar adresa de retur retinuta prin instructiunea **call**.

O schita simplista este prezentata în continuare.

```
.data
.text

proc1:

    ; push-uri si pop-uri in numar egal

    ret ; stiva arata ca la intrare in procedura,
        ; in varf se afla deci adresa instructiunii mov 1, eax
        ; salt la adresa din varf

.global main
main:

    call proc1 ; este adaugata pe stiva adresa lui mov 1, eax
               ; salt la proc

    mov $1, %eax
    xor %ebx, %ebx
    int $0x80
```

2.2 Argumentele unei proceduri

In continuare, pentru a transmite argumentele unei proceduri, acestea vor fi depozitate tot pe stiva. Datorita conceptului pe care il implementeaza o stiva - LIFO (Last In, First Out) este necesar ca argumentele sa fie stocate pe stiva in ordine inversa pentru a putea fi accesate in ordine naturala. Pe cazul general, consideram urmatorul apel **proc(arg1, arg2, ..., argn)**. Schita pentru apel este urmatoarea.

```
pushl argn
pushl argn-1
...
pushl arg2
```

```
pushl arg1
call proc
popl %eax
popl %eax
...
popl %eax
; n pop-uri corespunzatoare celor n argumente
```

Observatie: Toate argumentele incarcate pe stiva, trebuie eliminate la iesirea din procedura.

Exemplu: Fie acum functia **add** avand declaratia **add(x,y,&s)** care realizeaza adunarea a doi intregi **x** si **y** si depoziteaza suma in **s**. Pana in acest punct, un program minimalist in care se cheama respectiva functie cu argumentele **2, 3**, respectiv adresa unei variabile din memorie, este prezentat mai jos.

```
.data
x: .long 2
y: .long 3
s: .space 4
.text

add:
    movl 4(%esp), %eax
    addl 8(%esp), %eax
    movl 12(%esp),%ebx
    movl %eax, 0(%ebx)

    ret

.global main
main:

    pushl $s
    pushl y
    pushl x
    call add
    popl %edx
    popl %edx
    popl %edx

    mov $1, %eax
    xor %ebx, %ebx
    int $0x80
```

Cerinta: Rulati cu debuggerul si observati evolutia stivei. Explicati indicii de accesare si modul prin care este transmis un parametru cu referinta.

Observatie: Indicii de accesare sunt crescatori, ordinea argumentelor fiind cea descrisa in enunt.

2.3 Crearea cadrului de apel

In exemplul anterior a fost folosit pentru accesare elementelor de pe stiva registrul %esp. Totusi aceasta modalitate de lucru poate deveni foarte rapid incomoda daca pe parcursul procedurii apelate este repetat un numar de instructiuni de **push** si **pop**. Dupa fiecare astfel de operatie ar trebui ca programatorul sa regandeasca asezarea de pe stiva si sa schimbe modul de accesare. Practic e posibil ca un acelasi element sa fie accesat o data ca **0(%esp)**, iar dupa adaugarea unei alte valori pe stiva sa fie accesat ca **4(%esp)**. Intrucat dorim sa mentinem constant modul in care ne raportam la valori de pe stiva, ne vom raporta la registrul **%ebp**, depre care stiam ca memoreaza baza stivei.

Anterior, a fost mentionat, de asemenea, ca pentru fiecare apel de functie se aloca o zona de memorie pe stiva care este golita la iesirea din functie. O astfel de zona se numeste **stack frame** si practic delimiteaza zona apartinand unei proceduri. Pentru procedura curenta, varful stack frame-ului va fi reprezentat de valoarea lui **%esp**. Am dori sa avem si o marcare a bazei acestei zone, iar numele ne trimite direct cu gandul la **%ebp**. Ar fi ideal ca la executia in procedura curenta, **%ebp** sa indice baza zonei proprii de memorii de pe stiva si nu baza intregii stive care nu ar fi de folos intrucat nu suntem interesati de toate **stack frame**-urile create anterior din diverse apeluri imbricate.

Pentru aceasta la fiecare creare de cadru de apel, registrul **%ebp** va fi modificat pentru a indica baza **stack frame**-ului curent. Totusi acesta nu este un pas suficient pentru ca la revenirea din procedura, ne-am dori ca **%ebp** sa indice catre baza **stack frame**-ului procedurii apelante. Atunci o solutie evidenta este sa memoram valoarea initiala la intrarea in procedura si sa o restauram la iesire pentru a se mentine in registrul **%ebp** valoarea corespunzatoare a cadrului in orice moment al executiei.

Astfel, un cadru corect de apel pentru schema din subsectiunea 2.1 este urmatorul:

```
.data
.text

proc1:
    pushl %ebp
    movl %esp, %ebp

    ; codul din corpul procedurii

    popl %ebp
    ret

.global main
main:

    call proc1


    mov $1, %eax
    xor %ebx, %ebx
    int $0x80
```

Asadar, vom rescrie exemplul din subsectiunea 2.2 raportat la noile cunostinte. Pentru aceasta sa vedem cum arata stiva.

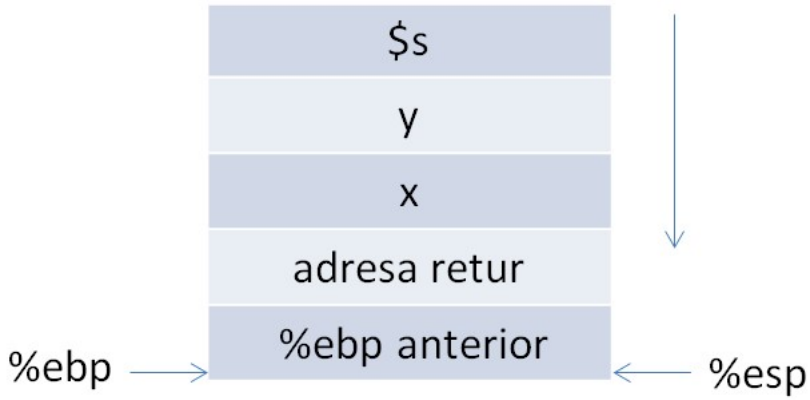


Figura 2: Stiva program subsectiunea 2.2

In acest context, in cadrul functiei **add** putem adauga oricate elemente dorim in jos in stiva pentru ca **%ebp** va ramane fix, singurul care isi modifica valoarea fiind **%esp**. Astfel, programul va fi rescris ca in exemplul de mai jos.

Exemplu:

```
.data
x: .long 2
y: .long 3
s: .space 4
.text

add:
    pushl %ebp
    mov %esp, %ebp

    movl 8(%ebp), %eax
    addl 12(%ebp), %eax
    movl 16(%ebp),%ebx
    movl %eax, 0(%ebx)

    popl %ebp
    ret

.global main
main:

    pushl $s
    pushl y
```

```
pushl x

call add

popl %edx
popl %edx
popl %edx

mov $1, %eax
xor %ebx, %ebx
int $0x80
```

Cerinta: Rulati cu debuggerul si observati din nou evolutia stivei, precum si noii indici de accesare.

Observatie: Variabilele locale din proceduri se salveaza tot pe stiva in continuarea cadrului descris anterior.

2.4 Registrii callee si caller saved

Dupa cum bine stim, pe ahitectura x86 numarul de registri este limitat. Am considerat abuziv pana acum ca printr-o analogie cu C-ul, registrii pot fi vazuti ca niste variabile ale programului. In acest caz, urmand aceeasi analogie, vom considera cazul in care avem o variabila **x** declarata in **main** si o variabila cu acelasi numele intr-o procedura apelata. Este bine-cunoscut ca practic cele 2 variabile nu se influenteaza una pe cealalta. Modificarea variabilei **x** din procedura nu are niciun efect asupra celei din **main**. In acelasi mod, trebuie ca registrii modificati intr-o procedura sa nu produca efecte secundare asupra utilizarii lor din apelant. Dorim sa regasim dupa intoarcerea din procedura, in toti registri folositi valorile depozitate inainte de apel.

Pentru aceasta avem din nou ca instrument principal stiva. Astfel in functie de locul in care sunt salvate valorile registrilor, exista 2 tipuri de registri:

- **callee-saved : %ebx, %esp, %ebp, %esi, %edi**. Valorile acestor registri se garanteaza a fi restaurate de catre procedura, adica acestea trebuie salvate in zona de variabile locale de catre procedura.
- **caller-saved : %eax, %ecx, %edx**. Nu este garantata restaurarea lor. Apelantul trebuie sa salveze aceste valori inainte de incarcarea argumentelor functiei daca doreste sa regaseasca valorile initiale la iesirea din functie.

In cele din urma, asadar, un cadru de apel va arata ca in figura 3

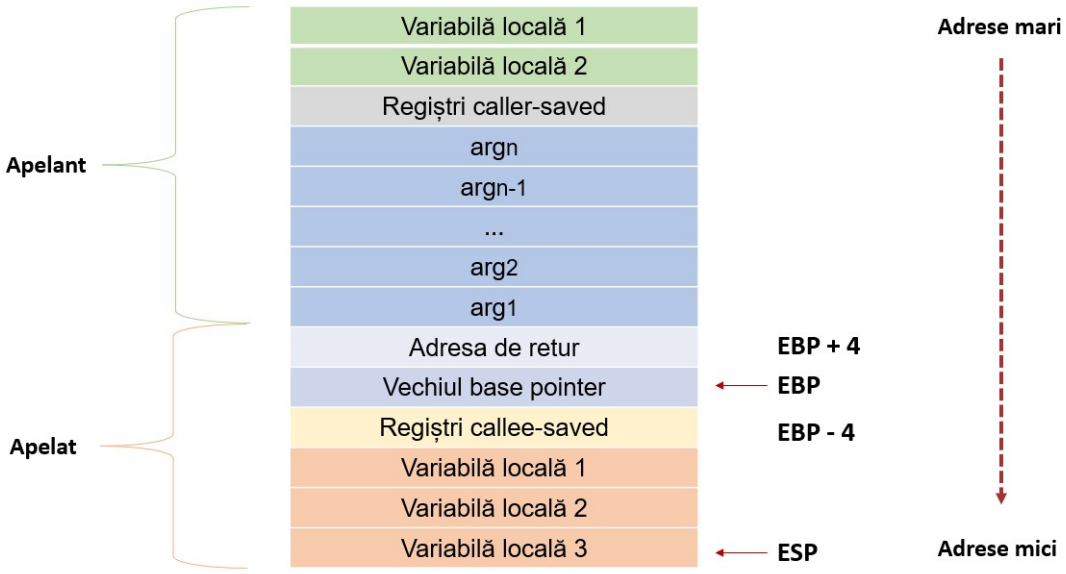


Figura 3: Cadru de apel

Exemplu: Sa luam acum acelasi cod pentru apelul functiei **add** si sa presupunem ca dupa apel exista o bucata de cod suplimentara care se asteapta sa reagaseasca in registrii **%eax** si **%ebx** valorile de dinainte de apel. O rescriere este prezentata in continuare.

```
.data
x: .long 2
y: .long 3
s: .space 4
.text

add:
    pushl %ebp
    mov %esp, %ebp

    pushl %ebx

    movl 8(%ebp), %eax
    addl 12(%ebp), %eax
    movl 16(%ebp),%ebx
    movl %eax, 0(%ebx)
```

```
    popl %ebx

    popl %ebp
    ret

.global main
main:

    movl $5, %eax
    movl $6, %ebx

    pushl %eax
    pushl $s
    pushl y
    pushl x
    call add

    popl %edx
    popl %edx
    popl %edx
    popl %eax

    eticheta:

    ; cod suplimentar

    mov $1, %eax
    xor %ebx, %ebx
    int $0x80
```

Cerinta: Rulati cu debuggerul si observati din nou evolutia stivei, precum si valorile registrilor %eax, %ebx inainte de inceperea bucatii de cod suplimentare.

2.5 Returnarea valorilor

Pana acum, am considerat numai proceduri care nu returnau nicio valoare. In cazul in care se doreste returnare, valorile vor fi depozitate in aceasta ordine in functie de numarul lor in **%eax, %ecx, %edx** si ulterior prin varful stivei.

Observatie: Registrii prin care se face returul sunt exact registrii care nu sunt restaurati de apelat. Totusi, si ei pot fi restaurati la un anumit punct in program daca valorile initiale erau necesare apelantului. Asadar, valorile returnate trebuie utilizate (sau memorate) inaintea unei astfel de instructiuni de restaurare a vechii valori.

Cerinta: Modificati ultimul program astfel incat procedura **add** sa aiba doar doi parametri si sa returneze suma in programul principal. Suma va fi ulterior incarcata in memorie la adresa lui **s**.

3 Apelul functiilor din C

Pentru a putea interactiona intr-un program dezvoltat in limbaj de asamblare cu functii din limbajul `C`, putem utiliza, in loc de *link*-area prezentata in primul suport de laborator, compilatorul `gcc`. In aceasta maniera, putem folosi functiile predefinite in libraria standard, printre care si cele mentionate anterior, si anume `printf`, `scanf` si `fflush`.

3.1 Apelul functiei `printf`

In limbajul `C`, functia `printf` este definita ca avand un numar variabil de argumente, primul fiind formatul `string`-ului de afisat. De exemplu, apeluri valide sunt urmatoarele:

```
printf("Numarul de afisat este: %ld\n", x);
printf("Numarul de afisat este: %d\n", x);
printf("Suma numerelor %hu si %hu este %hu\n", x, y, sum);
printf("Numele studentului cu cea mai mare medie este %s\n", nume);
```

Doua dintre formatele utile pe care le vom utiliza sunt

- `%d` - pentru afisarea int-urilor;
- `%ld` - pentru afisarea long-urilor;
- `%hu` - pentru afisarea short int-urilor (word-urilor);
- `%s` - pentru afisarea sirurilor de caractere.

Observatie! Long-urile și int-urile din C pe 32 de biți sunt echivalentul unui long in asamblare (ocupă 4 octeți)

Numarul argumentelor pe care le ia `printf` este dat de cate formate speciale utilizam in cadrul primului argument - de exemplu, pentru ca in sirul pentru afisarea sumei de la exemplul anterior avem trei aparitii ale lui `%ld`, inseamna ca numarul de argumente va fi egal cu 1 (sirul de format) + cate un argument pentru fiecare valoare care trebuie completata (3) = 4.

Pentru a apela functia `printf` intr-un program dezvoltat in limbaj de asamblare, vom proceda astfel: (presupunem ca urmatorul fisier este denumit `call_printf.s`):

```
.data
    x: .long 23
    formatString: .asciz "Numarul de afisat este: %ld"
.text

.global main

main:
    ; incarcam argumentele in ordinea inversa limbajului C
    ; pentru printf("Numarul de afisat este: %ld", x);
    ; incarcam x, apoi sirul de format
    pushl x
    pushl $formatString
```

```
    ; apelam functia printf
    call printf

    ; pentru fiecare push facut, facem un pop
    ; alegand un registru pe care nu-l utilizam
    popl %ebx
    popl %ebx

    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Observatii importante:

- pentru ca `printf` acceseaza argumentele in ordinea naturala - mai intai formatul sirului de afisat, apoi argumentele, in ordinea in care apar si in format, inseamna ca le vom incarca in **ordine inversa** pe stiva;
- formatul sirului **nu** se da ca valoare, ci **ca adresa in memorie**, de aceea trebuie ca numele lui sa fie prefixat de simbolul `$`;
- pentru a face `pop`, putem utiliza orice registru care nu contine, momentan, o valoare pe care vrem sa o pastram - `pop` modifica accesul pe care il are registrul `%esp` (*stack pointer*-ul) asupra stivei, si descarca valoarea actuala din *top* in registrul pe care noi il precizam. In acest caz, am ales sa facem descarcarea in `%ebx`, fiind echivalent orice alt registru;
- instructiunea `xorl %ebx, %ebx` este echivalenta cu instructiunea `movl $0, %ebx`.

Pentru a compila programul, vom utiliza compilatorul `gcc`:

```
gcc -m32 call_printf.s -o call_printf
./call_printf
```

Cel mai probabil, in acest moment nu vom avea o valoare efectiva afisata la **standard output**, din cauza modului in care sunt implementate sistemele de operare. Pentru a obliga afisarea *buffer*-ului curent de memorie, vom utiliza functia `fflush` cu argumentul `NULL`. Apelul este foarte simplu, trebuie sa incarcam pe stiva `NULL`, adica valoarea `0`, si sa o descarcam, din nou, intr-un registru pe care nu-l avem momentan completat cu o valoare utila - sa presupunem ca tot `%ebx`. Apelul va fi:

```
pushl $0
call fflush
popl %ebx
```

In acest moment, exemplul complet va avea urmatoarea forma:

```
.data
    x: .long 23
    formatString: .asciz "Numarul de afisat este: %ld"
.text
```

```
.global main

main:
    pushl x
    pushl $formatString
    call printf
    popl %ebx
    popl %ebx

    pushl $0
    call fflush
    popl %ebx

    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Vom compila iar:

```
gcc -m32 call_printf.s -o call_printf
./call_printf
```

Care, de aceasta data, va afisa corect pe ecran mesajul

```
Numarul de afisat este: 23
```

Observatie. Pentru a standardiza modul in care vom face *call*-urile catre `printf`, vom avea mereu un `printf` urmat de un `fflush`.

3.2 Apelul functiei `scanf`

Vom pleca, din nou, de la modul in care aceasta functie este apelata in limbajul `C`, si anume:

```
scanf("%ld", &x);
scanf("%ld %ld", &x, &y);
```

Desi apelul seamana foarte mult cu cel pe care il utilizam pentru `printf`, observam ca, de aceasta data, argumentele nu mai sunt date prin valoarea propriu-zisa - nu apelam `x`, ci `&x`. Acest lucru trebuie sa se reflecte si in limbajul de asamblare. Pentru aceasta, avem simbolul `$`, care ne da acces la locatia de memorie a unei variabile declarate in sectiunea `.data`.

Vom demonstra aceste lucruri prin intermediul codului de mai jos, presupus salvat intr-un fisier de forma `call_scanf.s`, pentru citirea unui intreg de la tastatura.

```
.data
    x: .space 4 ; nu stim momentan valoarea citita, dar stim cat ocupa
    formatString: .asciz "%ld"
.text

.global main
```

```
main:
    ; incarcam in stiva NU x, ci ADRESA lui x
    ; adica nu x, ci £x
    pushl $x
    ; incarcam adresa formatului
    pushl $formatString
    call scanf
    ; descarcam ce am incarcat in stiva
    popl %ebx
    popl %ebx

    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

3.3 Un exemplu de citire si de afisare a unui graf neorientat

Pentru a demonstra utilizarea pentru `printf`, `scanf` si `fflush`, vom prezenta citirea unui graf neorientat, dat prin numarul de varfuri, numarul de muchii si muchiile acestuia.

```
.data
        matrix: .space 1600
        columnIndex: .space 4
        lineIndex: .space 4
        n: .space 4
        nrMuchii: .space 4
        index: .space 4
        left: .space 4
        right: .space 4
        formatScanf: .asciz "%ld"
        formatPrintf: .asciz "%ld "
        newLine: .asciz "\n"
.text

.global main

main:
        pushl $n
        pushl $formatScanf
        call scanf
        popl %ebx
        popl %ebx

        pushl $nrMuchii
        pushl $formatScanf
        call scanf
        popl %ebx
        popl %ebx
```

```
    ; simulam, de fapt
    ; for (long index = 0; index < nrMuchii; index++)
    ; {
    ;       scanf("%ld", &left);
    ;       scanf("%ld", &right);
    ;       matrix[left][right] = 1;
    ;       matrix[right][left] = 1;
    ; }
        movl $0, index
et_for:
        movl index, %ecx
        cmp %ecx, nrMuchii
        je et_afis_matr

        pushl $left
        pushl $formatScanf
        call scanf
        popl %ebx
        popl %ebx

        pushl $right
        pushl $formatScanf
        call scanf
        popl %ebx
        popl %ebx

        ; trebuie sa completez
        ; matrix[left][right] = 1
        ; indexul este left * n + right

        movl left, %eax
        movl $0, %edx
        mull n
        ; %eax := left * n
        addl right, %eax
        ; %eax := left * n + right

        lea matrix, %edi
        movl $1, (%edi, %eax, 4)

        ; trebuie sa completez matrix[right][left] = 1
        ; indexul este right * n + left

        movl right, %eax
        movl $0, %edx
        mull n
        ; %eax := right * n
```

```
        addl left, %eax
        ; %eax := right * n + left

        lea matrix, %edi
        movl $1, (%edi, %eax, 4)

        incl index
        jmp et_for

et_afis_matr:
        movl $0, lineIndex
        for_lines:
                movl lineIndex, %ecx
                cmp %ecx, n
                je et_exit

                movl $0, columnIndex
                for_columns:
                        movl columnIndex, %ecx
                        cmp %ecx, n
                        je cont

                        ; afisez matrix[lineIndex][columnIndex]
                        ; indexul este lineIndex * n + columnIndex
                        movl lineIndex, %eax
                        movl $0, %edx
                        mull n
                        addl columnIndex, %eax
                        ; %eax = lineIndex * n + columnIndex

                        lea matrix, %edi
                        movl (%edi, %eax, 4), %ebx

                        pushl %ebx
                        pushl $formatPrintf
                        call printf
                        popl %ebx
                        popl %ebx

                        pushl $0
                        call fflush
                        popl %ebx

                        incl columnIndex
                        jmp for_columns

        cont:
                movl $4, %eax
```

```
            movl $1, %ebx
            movl $newLine, %ecx
            movl $2, %edx
            int $0x80

            incl lineIndex
            jmp for_lines


et_exit:
        movl $1, %eax
        movl $0, %ebx
        int $0x80
```

Nota: Testati-va cunostintele acumulate pana acum facand testele de laborator 6.1, 6.2 si 6.3 pe care le gasiti online la sectiunea laboratoare.

4 Manipularea numerelor în viruglă mobilă în x86 AT&T

Arhitectura x86 suporta operatii pe numere in virgula mobila, utilizand doua abordari, **FPU** (*Floating Point Unit*) si **SSE** (*Streaming SIMD Extensions*). In prima abordare, sunt utilizati registrii `st(x)`, iar in cea de-a doua, sunt utilizati registrii `xmm`.

Numerele in virgula mobila sunt reprezentate conform standardului *IEEE 754*, iar in cadrul laboratorului vom lucra doar cu formatul `float`, pe 32 biti (arhitectura suporta si un format `double`, pentru mai multa precizie, reprezentat pe 64 biti). In formatul `float`, avem 1 bit pentru semn, 8 biti pentru exponent, respectiv 23 biti pentru mantisa, iar codificarea unei valori este data de formula $\text{Val} := (-1)^S \cdot 2^{\text{E}-\text{Bias}} \cdot (1.\text{M})$.

In ceea ce priveste abordarile, respectiv registrii utilizati, pentru **FPU** avem registrii de la `st(0)` la `st(7)`, care sunt organizati intr-o stiva, iar instructiunile **FPU** opereaza implicit pe aceasta stiva, iar pentru **SSE** avem registrii de la `xmm0` la `xmm7`.

In cele ce urmeaza, prezentam instructiunile pe care le putem utiliza pentru a putea manipula numerele in virgula mobila.

Pentru **FPU**:

Instructiune	Operanzi	Scurta explicatie
`flds`	`mem`	Incarca un `float` in stiva FPU, in `st(0)`
`fldl`	`mem`	Incarca un `double` in stiva FPU, in `st(0)`
`fstps`	`mem`	Stocheaza un `float` din `st(0)` in memorie
`fstpl`	`mem`	Stocheaza un `double` din `st(0)` in memorie

Pentru **SSE**:

Instructiune	Operanzi	Scurta explicatie
`movss`	`src, dest`	Muta un scalar **float** de la **src** la **dest**
`addss`	`src, dest`	Aduna `float` din `src` cu `dest` si stocheaza in `dest`
`subss`	`src, dest`	Scade `src` din `dest` si stocheaza in `dest`
`mulss`	`src, dest`	Inmulteste `src` cu `dest` si stocheaza in `dest`
`divss`	`src, dest`	Imparte `dest` la `src` si stocheaza in `dest`
`sqrtss`	`src, dest`	Calculeaza `dest := sqrt(src)`
`cvtsi2ss`	`src, dest`	Converteste un `long` in `float` si pune rezultatul in `dest`
`maxss`	`src, dest`	Calculeaza maximul in `dest`
`minss`	`src, dest`	Calculeaza minimul in `dest`

Avem la dispozitie functii matematice in `libm`, biblioteca pe care o putem specifica in comanda de compilare, prin flag-ul `-lm`. (de exemplu `gcc -m32 float.s -o float -no-pie -lm`) Exemplu de functii pe care le putem utiliza: `logf`, `sqrtf`, `expf`. Conventia de apel se respecta, argumentele se incarca pe stiva in ordine inversa, iar rezultatul functiei este utilizat sau in `st(0)`, daca utilizam **FPU**, sau in `xmm0`, daca utilizam **SSE**.

Exemplu pentru calculul logaritmului natural:

```
.data
    number: .float 2.718281828
```

```
    logResult: .space 4
.text

.global main

main:
    flds number     // incarcam valoarea pe stiva FPU
    subl $4, %esp
    fstps 0(%esp)   // efectuam un push pentru a respecta conventia de apel

    call logf

    fstps logResult // am primit rezultatul in st(0), il descarcam si il salvam in logResult
    addl $4, %esp   // pop pe stiva

et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Rulam acest program (din fisierul numit, de exemplu, `float1.s`) cu `gcc -m32 float1.s -o float1 -no-pie -lm`. Utilizam `gdb` pentru a verifica daca s-a efectuat calculul corect: `b et_exit`, `run`, `print (float) logResult`. Observam ca obtinem valoarea `0.99999994`.

Oferim si un exemplu pentru determinarea mediei aritmetice a elementelor dintr-un array de `float`-uri:

```
.data
    v: .float 1.0, 2.0, 3.0, 4.0
    n: .long 4
    result: .space 4
    mem: .space 4
.text

.global main

main:
    lea v, %edi                 // adresa array-ului
    xorl %ecx, %ecx             // pornim de la indexul 0
    movl $0, mem                // utilizam memorie intermediara
    movss mem, %xmm0            // initializam suma cu 0

et_loop:
    cmpl n, %ecx
    je avg

    movss (%edi,%ecx,4), %xmm1  // incarcam elementul din array
    addss %xmm1, %xmm0          // il adaugam la suma
```

```
    incl %ecx
    jmp et_loop

avg:
    cvtsi2ss %ecx, %xmm1        // convertim numarul de elemente la float
    divss %xmm1, %xmm0          // impartim si salvam in xmm0 rezultatul

    movss %xmm0, result

et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Pentru a inspecta valoarea, consideram ca fisierul se numeste `float2.s`, si rulam `gcc -m32 float2.s -o float2 -no-pie -lm`, iar cu `gdb`: `b et_exit`, `run`, `print (float) result` si observam ca obtinem valoarea `2.5`.

Exercitiu. Considerati ca aveti un *array* de `float` care stocheaza probabilitatile $\{p_i\}_{i\in\{1,\ldots,n\}}$ a n evenimente. Determinati **entropia** acestei multimi. Implementati aceasta problema utilizand o procedura cu signatura `float entropy(float *probabilities, float n)`.

5 Exerciții

1. (a) Sa se defineasca procedura perfect(x), cu x numar natural. Un numar este perfect daca este egal cu suma divizorilor sai pana la jumatate. Exemplu: $6 = 1 + 2 + 3$; $28 = 1 + 2 + 4 + 7 + 14$;

   (b) Se dau de la tastatura un intreg `n` si un vector cu `n` elemente. Sa se afiseze pe ecran numarul de elemente perfecte.

2. Sa sa implementeze procedura x86 cu acelasi efect ca procedura C **memcpy**:

```
void *memcpy(void *dest, const void *src, size_t n);
```

   care copiaza **n** octeti de la adresa **src** la adresa **dest** si returneaza **dest**.

3. Scrieti o procedura ce implementeaza functia C **atoi**:

```
int atoi(const char *s);
```

   care returneaza intregul a carui reprezentare externa zecimala este continuta in stringul **s** sau 0 daca stringul nu poate fi convertit.

4. Translatati in limbaj de asamblare x86 urmatorul program C:

```
#include<stdarg.h>

void aduna(long *a, long n, ...){
    register long i;
    va_list l;
    va_start(l,n);
    *a=0;
    for(i=0;i<n;++i) *a+=va_arg(l, long);
    va_end(l);
}

long s, s1;

void main(){
    aduna(&s,3,1,2,3);     /* obtinem s=6 */
    aduna(&s1,2,10,20);    /* obtinem s1=30 */
}
```

5. Sa se implementeze un program care sa calculeze functia $f(x) = 2g(x)$, unde $g(x) = x + 1$.

6. Sa se implementeze procedura **proc(x)**, $x > 1$, cu definitia:

```
void proc(long x)  {
    printf("\%ld ",x);
    if (x != 0)
        proc(x-1);
}
```

7. Sa se transcrie in limbaj de asamblare x86:

```
long aduna(long a, long b) {
    return a+b;
}

void iteratie(long *a, long *b) {
    long c;
    c=aduna(*a, *b);
    (*a)=(*b); (*b)=c;
}

void main() {
    long n=5,x=1,y=1,z;
    register long i;
    for(i=2;i<n;++i) iteratie(&x,&y);
    z=y;
}
```

8. Sa se implementeze o procedura care calculeaza recursiv factorialul unui numar.

9. Scrieti un program care sa calculeze al n-lea termen (t_n) din sirul lui Fibonacci (folosind recursivitate). Afisati la final un text de forma *Al n-lea element din sirul lui Fibonacci este x*. Vom considera primele 2 elemente $t_0 = 0$ si $t_1 = 1$. Exemplu: $n = 5$ => Al 5-lea termen din sirul lui Fibonacci este 5.

10. Se introduc de la tastatura n si k ($k \leq n$). Sa se scrie o functie $C(n, k)$ recursiva ce calculeaza combinari de n luate cate k dupa formula:

$$C(n,k) = \begin{cases} 1 & \text{daca } k = 0 \\ 1 & \text{daca } k = n \\ C(n,k) = C(n-1,k) + C(n-1,k-1) & \text{altfel.} \end{cases}$$

6 Resurse suplimentare

Pentru mai multe detalii despre GNU Assembly va recomandam cartea *Professional Assembly Language* de Richard Blum si in special urmatoarele capitole:

- Chapter 7: Using numbers subcapitolele SIMD Integers, Floating-Point Numbers;
- Chapter 9: Advanced Math Functions;
- Chapter 11: Using Functions;
- Chapter 12: Using Linux System Calls;
- Chapter 13: Using Inline Assembly;
- Chapter 15: Optimizing Routines;
- Chapter 17: Using Advanced IA-32 Features.

Arhitectura Sistemelor de Calcul
Laborator - Partea 0x02

Ruxandra Bălucea
Bogdan Macovei
Cristian Rusu

Noiembrie 2024

Cuprins

1 Introducere **3**
1.1 Context . . . 3
1.2 Formatul instrucțiunilor și encodarea . . . 4
1.3 Extensii RISC-V . . . 5
1.4 Comparație x86 vs RISC-V . . . 6

2 Mediul de lucru **7**

3 Limbajul Assembly RISC-V **9**
3.1 Regiștrii arhitecturii . . . 9
3.2 Tipuri de date . . . 10
3.3 Instrucțiunile principale . . . 10
3.4 Arhitectură load-store . . . 11
3.5 Relocări . . . 12
3.6 Apeluri de sistem . . . 14
3.7 Salturi . . . 14
3.8 Tablouri unidimensionale . . . 16
3.9 RV32C - instrucțiuni comprimate . . . 16

4 Convenția de apel **18**
4.1 Apelul unei proceduri . . . 18
4.2 Argumentele unei proceduri și valoarea de retur . . . 19
4.3 Crearea cadrului de apel . . . 20

5 Pipelining **23**
5.1 Hazard-uri în pipeline . . . 24
5.2 Măsurarea performanței . . . 26

6 Cache **27**
6.1 Performanța memoriei cache . . . 30

7 Exercitii **31**
7.1 Laboratorul 9 31
7.2 Laboratorul 10 31
7.3 Laboratorul 11 31

8 Resurse suplimentare **32**

1 Introducere

1.1 Context

RISC-V este un set de instrucțiuni(**ISA** - *Instructions Set Architecture*) RISC dezvoltat în 2010 la Universitatea Berkeley din California ce prezintă următoarele caracteristici principale:

1. Este o arhitectură **open-source.** Spre deosebire de x86 și ARM, oricine poate utiliza și implementa RISC-V fără taxe de licențiere sau restricții, promovând inovarea și reducând dependența de furnizorii comerciali.

2. **Modularitate.** RISC-V definește un set de instrucțiuni de bază minimalist la care se pot adăuga extensii opționale (pentru operații în virgulă mobilă, vectori, criptografie, cache management etc).

3. **Portabilitate.** Arhitectura este neutră din punct de vedere hardware, ceea ce o face ușor de implementat pe diverse tipuri de dispozitive. RISC-V poate fi utilizat atât în procesoare simple de IoT, cât și în procesoare complexe pentru servere și supercomputere.

Comunitatea globală de dezvoltatori contribuie activ la standardizarea și implementarea RISC-V, iar organizații precum **RISC-V International**[1] coordonează aceste eforturi. Dintre membrii fondatori amintim: Andes, Antmicro, ETH Zurich, Google, IBM, lowRISC, Microchip, Qualcomm, Rambus, Rumble, SiFive etc.

Inovația accelerată, independența tehnologică și costurile reduse fac din RISC-V alegerea a numeroase companii pentru dispozitive embedded, mobile, centre de date și supercomputere, inteligență artificială, automotive și chiar aplicații militare și guvernamentale.

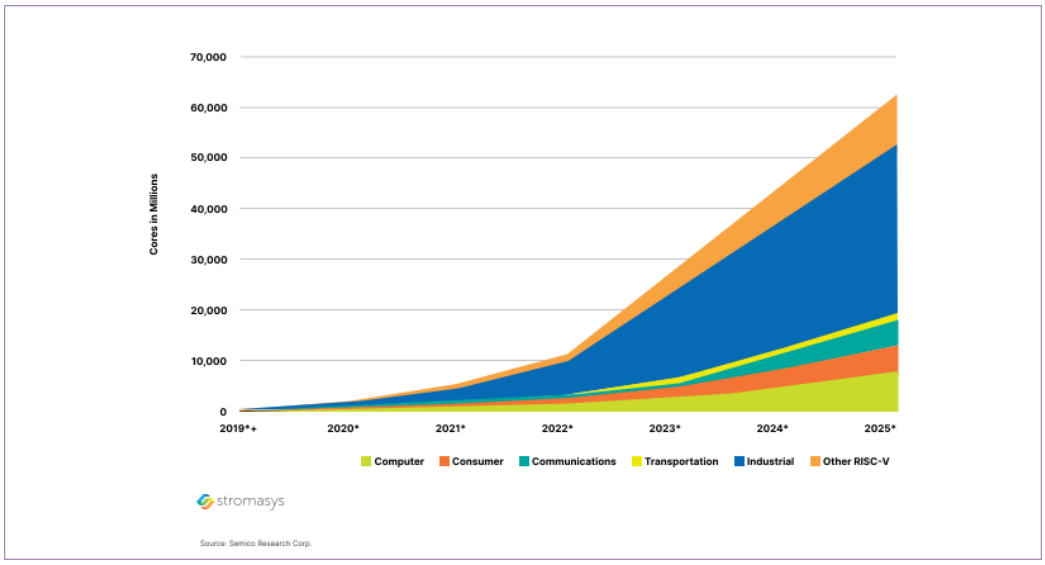


Figura 1: Evolutia RISC-V [2]

[1] https://riscv.org/
[2] https://www.stromasys.com/resources/all-about-the-risc-v-processors/

1.2 Formatul instrucțiunilor și encodarea

Arhitecturile RISC-V și x86 adoptă abordări fundamental diferite în ceea ce privește formarea instrucțiunilor și codificarea lor, reflectând filosofia generală a designului lor. În timp ce RISC-V este simplă și modulară, cu un set fix de formate de instrucțiuni, x86 este o arhitectură mai complexă și mai veche, cu codificări variabile.

Codificarea instrucțiunilor x86 ocupă între 1 și 15 octeți și poate include diferite componente ce fac decodarea mai lentă și mai complicată. Dintre acestea amintim:

- **prefixuri** (de exemplu pentru diferențierea dimensiunii registrului - **mov %eax, %ebx** are encodarea **89 c3**, pe cand **mov %ax, %bx**) are encodarea **66 89 c3**;
- **opcode**-uri (mnemonica) - specifică instrucțiunea principală;
- **moduri de adresare** - registru/memorie;
- **operanzi** suplimentari.

Pe de cealaltă parte, instrucțiunile RISC-V au lungime fixă de 32 de biți (4 bytes) și respectă una dintre cele câteva formate standard (de exemplu, R, I, S, B, U, J), ceea ce face decodarea rapidă și simplă pentru hardware.

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

Figura 2: Formatul instrucțiunilor RISC-V [3]

Exemplu: Se consideră instrucțiunea **add a0, a1, a2** care realizează suma dintre valorile regăsite în regiștrii **a1**, respectiv **a2** și o depune în **a0**. Aceasta este o instrucțiune de tip R, având definite în manualul cu specificațiile arhitecturii[4] **func7 = 0000000**, **func3 = 000** și **opcode = 0110011**. În plus știm codurile regiștrilor ca fiind următoarele: (în decimal) **a0 = 10**, **a1 = 11**, **a2 = 12**. Luând în considerare informațiile anterioare, encodarea instrucțiunii va fi următoarea:

func7	rs2	rs1	funct3	rd	opcode
0 0 0 0 | 0 0 0	0 | 1 1 0 0 |	0 1 0 1 | 1	0 0 0 |	0 1 0 1 | 0	0 1 1 | 0 0 1 1

Așadar, grupând câte 4 biți și covertind în hexa, o să obținem encodarea **0x00c58533**.

Exercițiu: Care este encodarea pentru instrucțiunea **addi a0, a1, 15** care este de tip **I** și realizează suma dintre imediatul **15** și valoarea din registrul sursă **a1**, depozitând valoarea în registrul destinație **a0** (vezi codurile regiștrilor mai sus)? De asemenea, se cunosc **func7 = 0000000**, **func3 = 000** și **opcode = 0010011**.

[3] https://five-embeddev.com/riscv-user-isa-manual/Priv-v1.12/instr-table.html
[4] https://riscv.org/wp-content/uploads/2017/05/riscv-spec-v2.2.pdf

1.3 Extensii RISC-V

Așa cum am prezentat anterior, una dintre principalele avantaje ale arhitecturii este modularitatea și împărțirea instrucțiunilor pe funcții specifice care să poată fi folosite sau nu în acord cu scopul procesorului.

La nucleul RISC-V se află setul de instrucțiuni de bază, cunoscut sub denumirea de **RV32I** pentru arhitecturi pe 32 de biți și **RV64I** pentru arhitecturi pe 64 de biți. Acestea includ un set minim de instrucțiuni necesar pentru a implementa o arhitectură de procesor funcțională. Caracteristicile principale ale bazei includ:

- Operații aritmetice de bază (add, sub etc);
- Operații de acces la memorie (lw, sw etc.);
- Instrucțiuni de control-flow (jal, beq etc)

De asemenea, dintre extensiile standardizate amintim:

- **RV32M** - suport pentru operații aritmetice mai avansate, cum ar fi multiplicarea (mul) și împărțirea (div). Este esențială pentru aplicații care necesită calcul numeric intensiv.
- **RV32C** - subset de instrucțiuni compresate, reducând dimensiunea codului și economisind memorie. Este utilă în sisteme integrate sau alte aplicații cu resurse limitate.
- **RV32A** - suport pentru operații atomice, care sunt cruciale în programele multithread. Instrucțiunile precum lr (load-reserved) și sc (store-conditional) permit sincronizarea eficientă între mai multe fire de execuție.
- **RV32F** și **RV32D** - suport pentru operații în virgulă mobilă. Acestea sunt esențiale în aplicații precum procesarea semnalelor digitale și învățarea automată.
- **RV32G** - este o extensie generală ce este echivalentă cu RV32IMAFD.
- **RV32V** - suport pentru calcul vectorial, esențial în aplicații precum procesarea imaginilor, simulările științifice și deep-learning.
- **RV32B** - instrucțiuni optimizate pentru manipularea biților, folosite în criptografie și procesarea semnalelor

La fel ca setul de bază, toate aceste extensii suportă și variante pe 64 de biți.

În plus, organizațiile și companiile pot defini propriile extensii pentru nevoi specifice, contribuind la diversitatea implementărilor RISC-V.

Toate extensiile ratificate, precum și statusul curent al extensiilor în curs de ratificare pot fi consultate aici[5].

[5] https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154732/Ratified+Extensions

1.4 Comparație x86 vs RISC-V

În finalul acestei secțiuni de introducere, prezentăm un tabel cu principalele avantaje și dezavantaje ale celor 2 arhitecturi studiate. Acest tabel poate fi privit chiar ca o comparație CISC vs RISC.

	x86 (CISC)	**RISC-V (RISC)**
Avantaje	• Codificare compactă pentru instrucțiuni simple → economisirea memoriei în aplicații vechi. • Suport pentru moduri complexe de adresare, util pentru software legacy.	• Simplitate și uniformitate → ușor de implementat în hardware și rapid de decodat. • Scalabilitate → extensiile pot fi adăugate fără a afecta codificările existente. • Pipeline eficient → formatele fixe permit instrucțiuni care pot fi decodate în paralel.
Dezavantaje	• Decodare lentă și complicată → dificultăți în proiectarea procesoarelor moderne. • Moștenirea istorică → dificultăți în a elimina instrucțiunile redundante sau ineficiente.	• Uneori necesită mai multe instrucțiuni pentru operațiuni complexe, comparativ cu x86.

Tabela 1: Comparație cu avantaje și dezavantaje între x86 și RISC-V

2 Mediul de lucru

Arhitectura RISC-V beneficiază de un ecosistem divers de tool-uri și simulatoare care acoperă nevoi educaționale, de cercetare și industriale. Mai jos sunt principalele tool-uri și simulatoare utilizate în ecosistemul RISC-V, împărțite în categorii relevante.

Simulatorul oficial este **Spike**[6] și este dezvoltat de RISC-V International, având suport pentru toate extensiile standard și putând fi folosit pentru testare și validare. Acesta ese destul de lent, principalul focus fiind pe acuratețe. Pe de cealaltă parte, **QEMU**[7] este un **emulator** și suportă mai multe arhitecturi (ARM, X86 etc.) și mai multe modalități de configurare de sistem, fiind potrivit pentru dezvoltarea de sisteme de operare și aplicații embedded. Permite integrarea de tool-uri precum **gdb** și este mult mai rapid, translatarea codului făcându-se dinamic din ISA-ul emulat în ISA-ul nativ al mașinii.

Un subdomeniu extrem de important pentru dezvoltarea ecosistemului RISC-V este reprezentat de design-ul hardware. Dintre cele mai cunoscute limbaje utilizate pentru proiectarea și validarea procesoarelor amintim **Chisel**[8] care permite definirea de scheme hardware pentru design-ul de procesoare. De asemenea, ca o platformă ce înglobează multe dintre procesoarele astfel dezvoltate, **Chipyard**[9] oferă suport pentru diverse core-uri (RocketChip, BOOM etc.) și acceleratoare, oferind de asemenea posibilitatea rulării în cloud pe un **FPGA** (circuit fizic reprogramabil pe baza unei scheme a unui sistem).

De asemenea, dintre cele mai imporante tool-uri software dezvoltate amintim:

- **RISC-V GNU Toolchain**[10] - compiler GCC, binutils, și GDB pentru RISC-V.
- **LLVM/Clang pentru RISC-V**[11] - compilare optimizată, suport extins pentru extensii ISA.

Majoritatea acestor ustensile de lucru sunt necesare a fi build-uite și adaptate pentru mediul de lucru al fiecăruia, proces ce poate dura și câteva ore. Din acest motiv, în cadrul laboratorului vom utiliza un simulator online cu o interfață user-friendly, simulator proiectat pentru utilizarea în scop educațional - **Ripes** - `https://ripes.me/`. Sursele pentru acest simulator care poate fi și instalat local se gasesc aici[12]. De asemenea, un tutorial amănunțit despre funcționalitățile simulatorului pot fi regăsite în secțiunea de documentație.

Acesta permite vizualizarea execuției instrucțiunilor într-un procesor simplificat, fie single-cycle, fie într-un pipeline. Oferă o interfață intuitivă pentru explorarea etapelor procesorului, modificarea regiștrilor, memoriei și stivei și rularea programelor în assembly RISC-V.

Ripes implementează extensia de bază RV32I, permițând și activarea extensiilor C și M.

Vom reveni asupra altor detalii despre acest simulator într-o secțiune ulterioară, punctând acum doar ce este necesar pentru a observa conceptele studiate anterior:

[6] `https://github.com/riscv-software-src/riscv-isa-sim`
[7] `https://www.qemu.org/`
[8] `https://www.chisel-lang.org/docs`
[9] `https://chipyard.readthedocs.io/en/stable/Chipyard-Basics/Chipyard-Components.html`
[10] `https://github.com/riscv-collab/riscv-gnu-toolchain`
[11] `https://github.com/llvm/llvm-project`
[12] `https://github.com/mortbopet/Ripes`

- **Secțiunile statice (.text, .data, .bss)** pot fi navigate prin butonul `Go to section:`.
- **Stiva** poate fi vizualizată prin folosirea butonului `Go to register:` și alegerea registrului **sp** care menține (similar cu esp), vârful stivei. Remarcăm poziționarea acesteia la o adresă mare 0x7fffff0, așa cum am discutat spațiul de adresă al unui proces în cadrul laboratorului despre proceduri pentru arhitectura x86.
- **Regiștrii** - aceștia pot fi vizualizați in tabelul din dreapta paginii.
- **Codul** - scrierea se face direct în fereastră putând observa o conversie instantă în binar, precum și varianta dezasamblată (encodată si ulterior decodată prin aplicarea succesivă a unui asamblor și a unui dezasamblor). De asemenea, se pot și încărca direct fișiere scrise local.

Atenție!

1. Pentru rularea instrucțiunilor pas cu pas, este nevoie sa schimbați procesorul din iconița de sub *File*, într-un **Single-cycle processor**.
2. Uneori site-ul refuza sa se încarce. Aceasta problema poate fi rezolvata prin curățarea memoriei cache sau prin deschiderea simulatorului într-o fereastră incognito.

3 Limbajul Assembly RISC-V

3.1 Regiștrii arhitecturii

RISC-V dispune de 32 de regiștrii generali (x0 până la x31), fiecare având o dimensiune de 32 de biți (pentru arhitectura de 32 de biți). Acești regiștri sunt utilizați pentru manipularea datelor, fiind folosiți la fel ca la x86 ca niște variabile.

Encodare	**Encodare comprimată**	**Registru**	**Nume ABI**	**Descriere**	**Callee-saved**
0	-	`x0`	`zero`	Hardwired zero	-
1	-	`x1`	`ra`	Return address	Da
2	-	`x2`	`sp`	Stack pointer	Da
3	-	`x3`	`gp`	Global pointer	-
4	-	`x4`	`tp`	Thread pointer	-
5	-	`x5`	`t0`	Temporary register 0	Nu
6	-	`x6`	`t1`	Temporary register 1	Nu
7	-	`x7`	`t2`	Temporary register 2	Nu
8	0	`x8`	`s0/fp`	Saved register 0 / frame pointer	Da
9	1	`x9`	`s1`	Saved register 1	Da
10	2	`x10`	`a0`	Function argument 0 / return value 0	Nu
11	3	`x11`	`a1`	Function argument 1 / return value 1	Nu
12	4	`x12`	`a2`	Function argument 2	Nu
13	5	`x13`	`a3`	Function argument 3	Nu
14	6	`x14`	`a4`	Function argument 4	Nu
15	7	`x15`	`a5`	Function argument 5	Nu
16	-	`x16`	`a6`	Function argument 6	Nu
17	-	`x17`	`a7`	Function argument 7	Nu
18	-	`x18`	`s2`	Saved register 2	Da
19	-	`x19`	`s3`	Saved register 3	Da
20	-	`x20`	`s4`	Saved register 4	Da
21	-	`x21`	`s5`	Saved register 5	Da
22	-	`x22`	`s6`	Saved register 6	Da
23	-	`x23`	`s7`	Saved register	Da
24	-	`x24`	`s8`	Saved register 8	Da
25	-	`x25`	`s9`	Saved register 9	Da
26	-	`x26`	`s10`	Saved register 10	Da
27	-	`x27`	`s11`	Saved register 11	Da
28	-	`x28`	`t3`	Temporary register 3	Nu
29	-	`x29`	`t4`	Temporary register 4	Nu
30	-	`x30`	`t5`	Temporary register 5	Nu
31	-	`x31`	`t6`	Temporary register 6	Nu

Tabela 2: Regiștrii de uz general RISC-V

Un alt registru de interes este registrul **pc (program counter)** care menține întotdeauna adresa instrucțiunii ce urmează a fi executată.

3.2 Tipuri de date

Tipurile principale de date pe care le vom utiliza sunt

1. **byte** - după cum îi spune și numele, ocupă 1 byte, adică 8 biți în memorie;
2. **halfword** - ocupă 2 bytes (16 de biți) și este utilizat pentru stocarea numerelor fracționare;
3. **word** - ocupă 4 bytes (32 biți) și este utilizat pentru stocarea întregilor. **long** ocupă tot 32 de biți.
4. **doubleword** - ocupa 8 bytes (64 biți);
5. **ascii**, **asciz** și **space** cu aceeași semnificație ca pe x86 (**space** nu este implementat de Ripes).

3.3 Instrucțiunile principale

Instrucțiune	Operanzi	Descriere
`add`	`rd, rs1, rs2`	Adună `rs1 + rs2` și salvează în `rd`.
`sub`	`rd, rs1, rs2`	Scade `rs2` din `rs1` și salvează în `rd`.
`mul`	`rd, rs1, rs2`	Înmulțește `rs1` cu `rs2` (M-extension).
`div`	`rd, rs1, rs2`	Împarte `rs1` la `rs2` (M-extension).
`rem`	`rd, rs1, rs2`	Restul împărțirii întregi între `rs1` și `rs2`.
`and`	`rd, rs1, rs2`	Realizează `and` logic între `rs1` și `rs2`.
`or`	`rd, rs1, rs2`	Realizează `or` logic între `rs1` și `rs2`.
`xor`	`rd, rs1, rs2`	Realizează `xor` logic între `rs1` și `rs2`.
`sll`	`rd, rs1, rs2`	Shift la stânga logic cu valoarea din `rs2`.
`srl`	`rd, rs1, rs2`	Shift la dreapta logic cu valoarea din `rs2`.
`addi`	`rd, rs1, imm`	Adună `rs1 + imm` și salvează în `rd`.
`slti`	`rd, rs1, imm`	Compară `rs1 < imm` și setează `rd` la 1 sau 0.
`andi`	`rd, rs1, imm`	Realizează `and` între `rs1` și `imm`.
`lui`	`rd, imm`	Încarcă imm pe cei 20 de biți superiori din `rd` (`rd = imm` $\ll$ `12`)
`auipc`	`rd, imm`	Încarcă imm pe cei 20 de biți superiori din `rd`, adunând la această valoare și pc (`rd = pc + (imm` $\ll$ `12)`).

Tabela 3: Principalele instrucțiuni RISC-V și operanzii lor

De asemenea, există și câteva pseudoinstrucțiuni importante. O pseudoinstrucțiune este o instrucțiune care poate fi folosită în asamblare, dar care nu are o encodare proprie, ci este de fapt convertită în una sau mai multe instrucțiuni clasice.

Pseudoinstrucție	**Instrucțiuni echivalente**	**Descriere**
`li rd, imm`	`lui rd, imm[31:12]` `addi rd, rd, imm[11:0]`	Încarcă valoarea imediată `imm` în registrul `rd`.
`la rd, label`	`lui rd, label[31:12]` `addi rd, rd, label[11:0]`	Încarcă adresa etichetei `label` în registrul `rd`.
`move rd, rs`	`addi rd, rs, 0`	Copiază valoarea din registrul `rs` în registrul `rd`.
`nop`	`addi x0, x0, 0`	Instrucțiune "no operation", nu face nimic.

Tabela 4: Principalele pseudoinstrucțiuni RISC-V

3.4 Arhitectură load-store

RISC-V este o arhitectură bazată pe principiul load-store, ceea ce înseamnă că toate operațiile de procesare a datelor trebuie să utilizeze exclusiv registre pentru calcule, iar memoria poate fi accesată doar prin instrucțiuni dedicate de încărcare (load) și stocare (store). Această abordare este esențială în designul arhitecturilor RISC (Reduced Instruction Set Computing).

Deoarece toate operațiile implică doar regiștri, numărul de etape în pipeline-ul procesorului este redus, ceea ce duce la o execuție mai eficientă a instrucțiunilor. Memoria este accesată doar în etapele de load și store, reducând complexitatea logicii de execuție.

Pentru încărcarea unei valori din memorie vom folosi instrucțiunea:

```
lw reg1, offset(reg2)
```

unde `reg2` reține o adresă de memorie, `offset` reprezintă numărul de bytes adăugați pentru a ajunge pe poziția din care vrem să citim, iar `reg1` este registrul în care se va încărca valoarea din memorie. Pentru stocarea unei valori în memorie vom folosi instrucțiunea:

```
sw reg1, offset(reg2)
```

unde `offset` și `reg2` sunt folosiți similar pentru a calcula adresa din memorie la care se face stocarea, iar `reg1` este registrul din care se ia valoarea ce urmează a fi depozitată.

Aceste variante de instrucțiuni se aplică pe o dimensiune de 32 de biți. Similar există instrucțiuni pentru a lucra cu 16 biți (`lh` și `sh`), respectiv cu 8 biți (`lb` și `sb`).

Registrul `gp` este folosit pentru a reține adresa secțiunii `.data`. Una dintre modalitățile de accesare a datelor din memorie, este așadar prin folosirea unui offset raportat la baza secțiunii.

Exercițiu: Pentru o exemplificare, să considerăm secvența următoare de cod. Cum va arăta memoria în urma execuției celor 2 instrucțiuni? Ce observați legat de declarațiile din zona `.data`?

```
.data
    .long 5
    .long 6

.text
.global main
main:
    lw a0, 0(gp)
    sw a0, 4(gp)
```

3.5 Relocări

Pentru a putea folosi în continuare simbolii declarați în memorie în cadrul instrucțiunilor și a nu fi nevoiți să calculăm de fiecare offset-ul pentru accesarea datelor, este nevoie să ne uităm peste conceptul de relocare.

O relocare este un proces utilizat în timpul compilării, linkării sau încărcării unui program pentru a ajusta adresele de memorie sau referințele la simboluri, astfel încât programul să poată fi executat corect indiferent de poziția sa în memorie. Majoritatea operanzilor de tip imediat acceptă o variantă de relocare pentru deducerea ulterioară a respectivului offset. Dacă poziționarea secțiunilor este statică și simbolii sunt prezenți în sursa curentă, adresa lor se poate deduce în timpul procesului de compilare, adresa fiind calculată automat. Dacă simbolii în schimb sunt necunoscuți și sunt aflați în timpul procesului de linkare (sunt declarați în alte surse, biblioteci etc), iar pozițiile secțiunilor sunt fixe (în special pentru dispozitivele embedded unde sunt predefinite printr-un script de configurare), adresa este calculată în această etapă. Altfel, în cazul în care secțiunile sunt dinamice (poziționate la diferite adrese la fiecare rulare), relocările vor fi rezolvate abia la rulare.

Pentru utilizarea simboluri din zonele de date, RISC-V folosește următoarele relocări:

- **Relocări pentru partea superioară (%hi și %pcrel_hi)**. Cele două sunt folosite în cadrul instrucțiunilor `lui`, respectiv `auipc` pentru a prelua cei 20 de biți superior dintr-o adresă.
- **Relocări pentru offset-uri (%lo și %pcrel_lo)**. Cele două sunt folosite pentru instrucțiunile `lw` și `sw` pentru adăugarea offset-ului de 12 biți rămași. De asemenea, poate fi folosită și în cadrul unei instrucțiuni `addi` pentru a ajuta la calcularea adresei de început (pentru un vector, un string). Această relocare lo este precedată întotdeauna de o instrucțiune cu relocare hi, obținând prin această combinare adresa pe 32 de biți.
- **Relocări pentru salturi** - vor fi prezentate în secțiunea despre salturi.
- Altele.

De asemenea, o diferențiere importantă o reprezintă modul în care funcționează relocările absolute, respectiv relocările relative la program counter:

- **%lo(sym)**, respectiv **%hi(sym)** vizează exact biții inferiori, respectiv biții superiori ai adresei simbolului `sym`;

- **%pcrel_hi(sym)** face referire `(adr_sym - pc) >> 12` (biții superiori);
- **%pcrel_lo(label)** unde `label` indică spre o instrucțiune având o relocare de tip `%pcrel_hi(sym)` va considera `(adr_sym - label) % 2`^{12} (biții inferiori).

Astfel, pentru a obține adresa unei variabile, putem folosi una dintre variantele de mai jos:

```
.data
    a: .long 5

.text
.global main
main:
    # Varianta 1 (statică)
    lui a0, %hi(a)
    lw a1, %lo(a)(a0)

    # Varianta 2 (relativă cu relocări)
    l0:
    auipc a2, %pcrel_hi(a)
    lw a3, %pcrel_lo(l0)(a2)

    # Varianta 3 (convenție de scriere care se expandează la varianta 2)
    lw a4, a

    # Varianta 4 (relativă cu global pointer)
    lw a5, 0(gp)
```

Observație! Programul funcționează și fără un apel `exit(0)` în cazul acestui simulator, neexistând un sistem de operare.

Întrucât pentru Ripes cunoaștem de la început locația secțiunilor `.data` și `.text` și de asemenea simbolul `a` este în același fișier, putem aplica toate variantele de mai sus. Vom alege în continuare să folosim varianta 3 pentru programele pe care le vom avea de implementat. **Relocările relative sunt într-o variantă instabilă pe Ripes și nu vor funcționa corect!**

Exercițiu: Analizați cele 4 variante, observând ce adrese sunt reținute la fiecare pas. Care este diferența dintre relocările %hi și %pcrel_hi, respectiv %lo și %pcrel_lo? De ce variantele relative pot fi utilizate și când nu se cunosc static adresele secțiunilor? Observați comportamentul greșit al variantei 2 (comparativ cu varianta 3).

Exercițiu: Transformați variantele 1, 2 și 4 de mai sus astfel încât în regiștrii `a1, a3 și a5` să obțineți adresa lui `a` (în loc de valoarea lui) folosind instrucțiunea `addi`.

Exercițiu: Transformați variantele de mai sus pentru stocare și verificați ce variantă este corect implementată de Ripes. **Observație!** Toate variantele de mai sus sunt corecte arhitectural vorbind, stadiul dezvoltării tool-ului fiind singurul imp ediment în folosirea lor.

3.6 Apeluri de sistem

Pentru folosirea apelurilor de sistem mecanismul este similar cu ce am văzut pe x86, diferind doar regiștrii. Astfel, pentru a apela o funcție sistem vom folosi instrucțiunea

```
ecall
```

care va produce verificarea regiștrilor următori:

- `a7` - pentru stocarea codului funcției
- `a0 - a6` - pentru încărcarea argumentelor.

De exemplu, funcția `exit` are codul 93. Deci pentru realizarea apelui `exit(0)`, vom avea secvența următoare de mai jos.

```
li a7, 93
li a0, 0
ecall
```

Codurile de apel pentru **Ripes** pot fi găsite în secțiunea **Help** > **System calls**.

Exercițiu: Declarați un string și un întreg în memorie și afișați-le pe ecran.

3.7 Salturi

O instrucțiune importantă pentru efectuarea de salturi necondiționate este instrucțiunea:

```
jal rd, imm
```

care stochează valoarea `pc + 4` în registrul `rd` și face saltul la adresa `pc + imm`.

În general pentru executarea de salturi necondiționate vom folosi:

```
j label
```

care este de fapt o pseudoinstrucțiune pentru instrucțiunea:

```
jal x0, label
```

Cum registrul `x0` nu poate fi modificat, asta înseamnă că valoarea `pc + 4` nu se salvează nicăieri, iar saltul se face la adresa lui `label`.

Fiind vorba din nou despre lucratul cu simboli, avem și în acest caz o relocare specifică instrucțiunii - **R_RISCV_JAL** care indică înlocuirea simbolului cu adresa sa relativă la `pc`.

Pentru salturile condiționate, spre deosebire de x86, RISC-V comasează instrucțiunea de comparare și de salt într-una singură.

În mod similar, putem folosi în loc de imediat un simbol a cărui adresă va fi calculată relativ prin rezolvarea relocării **R_RISCV_BRANCH**.

Să considerăm următorul exemplu:

Instrucțiune	Operanzi	Descriere
`blt(u)`	`rs1, rs2, imm`	Dacă `rs1` $<$ `rs2`, salt la `pc + imm`
`ble(u)`	`rs1, rs2, imm`	Dacă `rs1` $<=$ `rs2`, salt la `pc + imm`
`bgt(u)`	`rs1, rs2, imm`	Dacă `rs1` $>$ `rs2`, salt la `pc + imm`
`bge(u)`	`rs1, rs2, imm`	Dacă `rs1` $>=$ `rs2`, salt la `pc + imm`
`beq`	`rs1, rs2, imm`	Dacă `rs1` $==$ `rs2`, salt la `pc + imm`
`bne`	`rs1, rs2, imm`	Dacă `rs1` $!=$ `rs2`, salt la `pc + imm`

Tabela 5: Principalele instrucțiuni RISC-V pentru salturi condiționate.

Pseudoinstrucțiune	Operanzi	Descriere
`beqz`	`rs1, imm`	Dacă `rs1` $==$ `0`, salt la `pc + imm`
`bnez`	`rs1, imm`	Dacă `rs1` $!=$ `0`, salt la `pc + imm`

Tabela 6: Principalele pseudoinstrucțiuni RISC-V pentru salturi condiționate.

```
.data
 .long 6
 .long 7
 egale: .asciz "Numerele sunt egale\n"
 diferite: .asciz "Numerele sunt diferite\n"
.text
.global main
main:

  lw a0, 0(gp)
  lw a1, 4(gp)

  beq a0, a1, l0
  li a7, 4
  la a0, diferite
  ecall
  j final

l0:
  li a7, 4
  la a0, egale
  ecall

final:
  li a7, 93
  li a0, 0
  ecall
```

Exercițiu: Analizați programul și conversiile simbolilor în imediați.

3.8 Tablouri unidimensionale

Mecanismul de funcționare este același ca pe x86, dar nu există o grupare specifică pentru accesarea elementelor.

Pentru clarificare, să considerăm următorul exemplu care realizează suma elementelor dintr-un vector de întregi și o depozitează în memorie:

```
.data
    n: .long 5
    s: .long 0
    v: .long 1, 2, 3, 4, 5

.text
.global main
main:
    lw a0, 0(gp)     # se incarca n in a0
    addi a1, gp, 8   # se pune adresa lui v in a1
    li a2, 0         # in a2 vom calcula suma
    li a3, 0         # in a3 vom mentine indexul

    begin:
    bge a3, a0, final

    slli a4, a3, 2   # a4 = 4 * index
    add a4, a4, a1   # a4 = adresa lui v + 4 * index
    lw a4, 0(a4)     # a4 = *(adresa lui v + 4 * index)
    add a2, a2, a4   # adunam elementul la suma totala
    addi a3, a3, 1   # incrementam indexul
    j begin

    final:
    sw a2, 4(gp)

    li a7, 93
    li a0, 0
    ecall
```

Exercițiu: Să se rescrie exemplul anterior, modificând întâi valoarea lui `n` în `n * 4` și incrementând apoi indexul cu `4` la fiecare iterație.

3.9 RV32C - instrucțiuni comprimate

Pentru a îmbunătăți performanța și eficiența memoriei, RISC-V include și un subset de instrucțiuni comprimate, care permit utilizarea a 16 biți pentru a reprezenta anumite instrucțiuni, reducând astfel dimensiunea programelor și economisind lățimea de bandă a memoriei.

RVC realizează o schemă simplă de conversie care permite scurtarea instrucțiunilor din setul de bază când:

1. imediatul/offsetul este suficient de mic;
2. unul dintre regiștri este x0 (zero register), x1 (return address) sau x2 (stack pointer-ul);
3. este folosit același registru pentru destinație și pentru sursă;
4. se folosesc regiștrii care au o varianta comprimabilă (vezi Tabelul 2)

Exemple

```
lw t0, 4(sp)     --> c.lwsp t0, 4(sp)    // este folosit registrul sp
addi t0, t0, 1   --> c.addi t0, 1        // sursa == destinatie == t0
lw a0, 4(s0)     --> c.lw a0, 4(s0)      // a0 si s0 au variante comprimabile
lw t0, 516(sp)   --> nu se comprima      // imediatul este prea mare
```

Astfel, sunt definite următoarele formate pentru encodarea instrucțiunilor comprimate:

Format	Meaning	15 14 13	12	11 10 9	8 7	6 5	4 3 2	1 0
CR	Register	funct4		rd/rs1		rs2		op
CI	Immediate	funct3	imm	rd/rs1		imm		op
CSS	Stack-relative Store	funct3	imm			rs2		op
CIW	Wide Immediate	funct3	imm				rd′	op
CL	Load	funct3	imm	rs1′		imm	rd′	op
CS	Store	funct3	imm	rs1′		imm	rs2′	op
CB	Branch	funct3	offset	rs1′		offset		op
CJ	Jump	funct3	jump target					op

Figura 3: Formatul instrucțiunilor RVC [13]

Opcode-urile acestor operații au fost gândite astfel încât să nu se suprapună cu cele ale instrucțiunilor uzuale. Sunt verificați parte dintre biții de identificare, iar procesorul trimite instrucțiunea către un decodor specializat care interpretează această varianta pe 16 biți. În rest, pipelinul rămâne neschimbat.

Principalele instrucțiuni sunt indicate mai jos. De asemenea, un tabel cu toate instrucțiunile comprimate poate fi găsit aici [14]

Pentru activarea acestor instrucțiuni în Ripes este nevoie să bifați și extensia C în meniul de unde ați ales tipul de procesor.

Exercițiu: Să se rescrie exemplul din Subsecțiunea 3.8, parcurgând șirul de la final spre început și folosind ca index inițial `4 * (n - 1)`. Ce dimensiune are zona **.text**?

Exercițiu: Să se rescrie codul anterior folosind instrucțiuni comprimate. Care este noua dimensiune a zonei de cod?

[13] https://www2.eecs.berkeley.edu/Pubs/TechRpts/2015/EECS-2015-209.pdf
[14] https://five-embeddev.com/riscv-user-isa-manual/Priv-v1.12/rvc-instr-table.html

Instrucțiune	Operanzi	Descriere
`c.addi`	`rd, imm`	Adaugă un imediat la registrul `rd`.
`c.addisp`	`rd, imm`	Adaugă un imediat (16 biți) la registrul `sp` (pointerul de stivă).
`c.add`	`rd, rs`	Adaugă valoarea din registrul `rs` la registrul `rd`.
`c.lwsp`	`rd, offset(sp)`	Încarcă un cuvânt din memorie la adresa `sp + offset` în registrul `rd`.
`c.swsp`	`rd, offset(sp)`	Stochează un cuvânt din registrul `rd` la adresa `sp + offset`.
`c.li`	`rd, imm`	Încărcă un imediat de 16 biți în registrul `rd`.
`c.j`	`offset`	Salt necondiționat la adresa specificată de `offset`.
`c.beqz`	`rs, offset`	Salt condiționat dacă registrul `rs` este zero.
`c.bnez`	`rs, offset`	Salt condiționat dacă registrul `rs` nu este zero.
`c.not`	`rd, rs`	Neagă (complementul pe biți) valoarea din registrul `rs` și o stochează în registrul `rd`.
`c.lw`	`rd', offset(rs')`	Încarcă un cuvânt din memorie la adresa `rs' + offset` în registrul `rd'`.
`c.sw`	`rd', offset(rs')`	Stochează un cuvânt din registrul `rd'` la adresa `rs' + offset`.

Tabela 7: Principalele instrucțiuni RVC și operanzii lor

4 Convenția de apel

Pentru alocarea de spațiu pe stivă este necesar să folosim instrucțiuni aritmetice și de operare cu memoria. Deși nu există instrucțiuni precum `push` și `pop`, comportamentul este același.

De exemplu, pentru a stoca valoarea din registrul `s0` pe stivă, vom folosi următoarea secvență:

```
addi sp, sp, -4
sw s0, 0(sp)
```

În sens invers, pentru a scoate elementul din vârful stivei și a îl depozita în registrul `s0`, vom folosi:

```
lw s0, 0(sp)
addi sp, sp, 4
```

4.1 Apelul unei proceduri

Pentru a apela o procedură, putem folosi două instrucțiuni:

```
call procedura
jal procedura
```

Aceasta este de fapt o pseudoinstrucțiune pentru instrucțiunea:

```
jal x1, procedura
```

care, ne reamintim, salvează adresa instrucțiunii următoare în registrul `x1`, adică registrul `ra` (return address). Așa cum îi spune și numele acest registru este utilizat pentru reținerea adresei de retur. Așadar spre deosebire de x86, revenirea dintr-o funcție se face bazat pe un registru, valoarea nemaifiind transmisă prin stivă. Returul se face prin instrucțiunea:

```
ret
```

RISC-V mai are o pseudoinstrucțiune de salt pe care nu am prezentat-o și anume:

```
jr reg
```

care produce saltul la adresa reținută în registrul `reg` și este la rândul ei o pseudoinstrucțiune pentru:

```
jalr x0, reg, 0
```

primul operand marcând la fel ca în cazul instrucțiunii `jal` locația în care se va stoca adresa instrucțiunii următoare, iar ultimul operand reprezentând offsetul.

Cum este destul de ușor de remarcat, instrucțiunea de retur este deci doar un alias pentru:

```
jr ra
```

Așadar ca schelet de bază pentru apelul unei proceduri vom avea următoarea secvență:

```
.data
.text
.global main
main:
    call proc    // jal proc
                 // jal ra, proc

    li a7, 93
    li a0, 0
    ecall

proc:

    ret          // jr ra
                 // jalr x0, ra, 0
```

Atenție! Pare că Ripes începe execuția întotdeauna în prima procedură pe care o găsește, motiv pentru care vom pune mereu funcția `main` prima.

4.2 Argumentele unei proceduri și valoarea de retur

În cazul RISC-V, de asemenea, argumentele sunt transmise și ele prin regiștrii conform convenției pe care am văzut-o și la apelurile de sistem. Regiștrii `a0 - a7` sunt folosiți pentru transmiterea primelor 8 argumente. Dacă există mai mult de 8 argumente, acestea vor fi depozitate pe stivă ca în cazul x86.

De asemenea, valorile returnate se vor menține în regiștrii `a0`, `a1`, iar în cazul în care sunt mai mult de două, vor fi transmise prin vârful stivei.

Exemplu: Fie acum funcția având declarația `proc(x,y,&sum)` care realizează adunarea a doi întregi `x` și `y` și depozitează suma în `sum` și care de asemenea returnează diferența celor două numere. Până în acest punct, un program minimalist în care se cheamă respectiva funcție cu argumentele 2, 3, respectiv adresa unei variabile din memorie, este prezentat mai jos.

```
.data
    x: .long 3
    y: .long 2
    sum: .long 0
    dif: .long 0

.text
.global main
main:
    lw a0, 0(gp)
    lw a1, 4(gp)
    addi a2, gp, 8
    call proc

    sw a0, 12(gp)

    li a7, 93
    li a0, 0
    ecall

proc:
    add a3, a0, a1
    sw a3, 0(a2)
    sub a0, a0, a1

    ret
```

4.3 Crearea cadrului de apel

La fel ca în cazul arhitecturii x86, avem nevoie să marcăm pe stivă acele locații de unde încep să se pună pe stivă elemente din cadrul curent. Registrul folosit pentru a marca începutul acestui "frame" este `fp` (frame pointer-ul). De asemenea, pentru a nu pierde adresa de retur în cazul unor apeluri imbricate, la intrarea în funcție se salvează și registrul `ra` (ambii sunt de fapt regiștrii callee-saved). `fp` este mutat pentru a pointa către acea locație de pe stivă de după aceste două salvări (Ripes acceptă doar denumirea de `s0`.

Astfel o procedură va avea următorul schelet:

```
proc:
    addi sp, sp, -8
    sw ra, 4(sp)
    sw s0, 0(sp)
    addi s0, sp, 0
```

```
    // corpul procedurii

    lw s0, 0(sp)
    lw ra, 4(sp)
    addi sp, sp, 8
    ret
```

În plus, conceptul de regiștrii callee-saved/caller-saved funcționează la fel ca pentru x86. Tipul fiecărui registru este menționat în Tabelul 2.

Exercițiu: Observați valorile depozitate în `s3`, respectiv `t5` la finalul execuției programului. Adăugați instrucțiunile necesare pentru a menține și valoarea registrului calleer-saved `t5`.

```
.data
    x: .long 3
    y: .long 2
    sum: .long 0
    dif: .long 0

.text
.global main
main:
    li s3, 10
    li t5, 11

    lw a0, 0(gp)
    lw a1, 4(gp)
    addi a2, gp, 8
    addi a3, gp, 12
    call proc

  final:
    li a7, 93
    li a0, 0
    ecall

proc:
    addi sp, sp, -8
    sw ra, 4(sp)
    sw s0, 0(sp)
    addi s0, sp, 0
    addi sp, sp, -4
    sw s3, -4(s0)

    add s3, a0, a1
    sw s3, 0(a2)

    sub t5, a0, a1
```

```
sw t5, 0(a3)

lw s3, -4(s0)
addi sp, sp, 4

lw s0, 0(sp)
lw ra, 4(sp)
addi sp, sp, 8
ret
```

Exercițiu: Este posibil să facem alocarea de spațiu pe stivă o singură dată, numai că offseții pentru salvarea lui `ra` și `fp` se modifică. Adaptați programul astfel încât să alocați de la început 12 octeți, iar la final să îi ștergeți.

Mai multe informații despre convențiile de apel puteți găsi aici [15].

[15] https://d3s.mff.cuni.cz/files/teaching/nswi200/202324/doc/riscv-abi.pdf

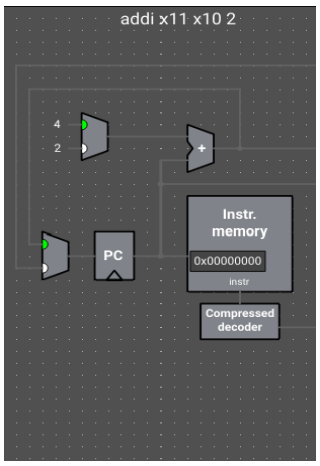
5 Pipelining

Pipeline-ul unui procesor este o tehnică utilizată pentru a spori performanța procesorului, permițând execuția simultană a mai multor instrucțiuni în etape diferite. .

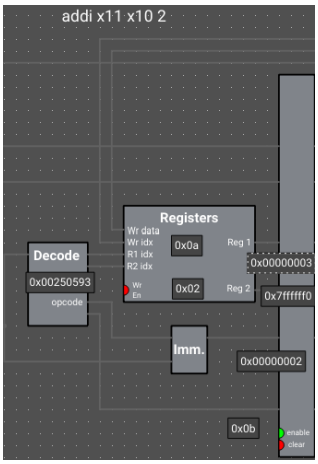
Pipeline-ul este o componentă esențială în arhitectura procesoarelor moderne, fiind prezent în procesoarele RISC (Reduced Instruction Set Computing) și CISC (Complex Instruction Set Computing). Până acum am folosit Ripes având în spate o simulare a unui proces într-un singur ciclu, în procesor putând exista la un anumit moment doar o singură instrucțiune. În continuare vom analiza un procesor în 5 "stage"-uri (primul dintre ele **5-stage processor w/o forwarding or hazard detection**) care este compus din următoarele unități:

- **Fetch (F)**: Se preia instrucțiunea din memorie.
- **Decode (D)**: Se decodează instrucțiunea pentru a determina ce operație trebuie executată.
- **Execute (E)**: Se execută operația pe baza instrucțiunii folosind unitatea aritmetico-logică.
- **Memory (M)**: Citirea sau scrierea datelor în memoria principală, dacă instrucțiunea implică astfel de operații.
- **Write Back (WB)**: Salvarea rezultatelor în registru sau memorie, pregătindu-le pentru instrucțiunile următoare.

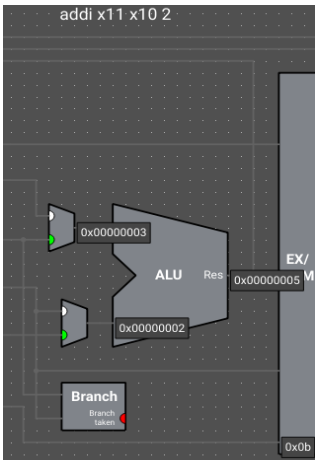
Să considerăm acum un program având o singură instrucțiune `addi a1, a0, 3`, iar `a0 = 2`. Să analizăm în continuare fiecare stage.



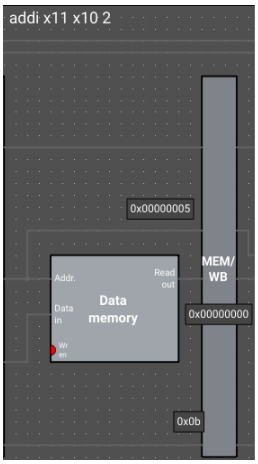
1. În etapa de **fetch**, prin PC (care este 0), este preluată instrucțiunea regăsită la acea adresă în memorie. Instrucțiunile comprimate sunt transformate în corespondentele lor necomprimate. Tot aici PC este incrementat cu 2 sau 4 pe baza acestei informații.



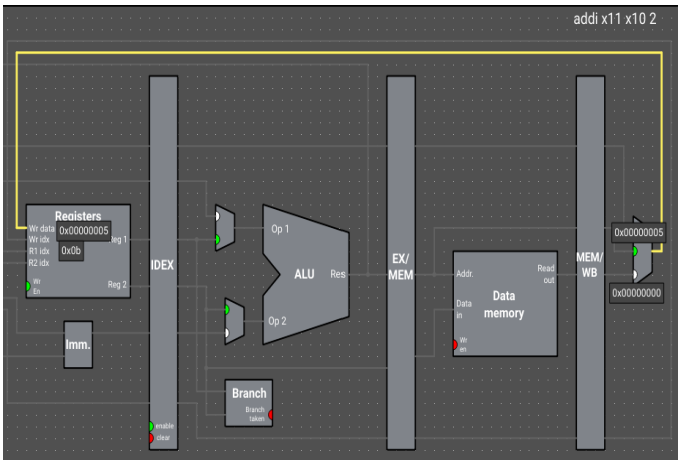
2. În etapa de **decode**, este interpretată encodarea și sunt preluate sursele și destinația operației. Sunt transmise mai departe valorile curente ale celor 2 regiștri sursă (sau 0 dacă nu există un al doilea), respectiv valoarea imediată și encodarea pentru identificarea ulterioară a operației.



3. În etapa de **execute**, cele două valori sunt transmise către ALU unde se aplică operația indicată de către opcode (adunarea cu imediat). Aceasta este o schemă simplificată, pentru a observa și ruta pe care se transmite tipul operației, puteți selecta o variantă de schemă extinsă.



4. În etapa de **memory**, se accesează datele din memorie (dacă ele există). După cum se poate observa, instrucțiunea aceasta nu folosește nimic din memorie, deci se trece prin în această etapă doar transmițând mai departe rezultatul.



5. În etapa de **write back** se produce scrierea efectivă în destinație - fie în memorie (în cazul nostru este 0, nu se accesează memoria), fie în register file, modificându-se valoarea registrului destinație.

În ciuda avantajelor, performanța unui pipeline depinde de mai mulți factori care pot introduce penalizări.

5.1 Hazard-uri în pipeline

Un hazard este o problemă care împiedică procesorul să execute eficient instrucțiunile în pipeline. Există trei tipuri principale:

1. **Hazarduri de date** - Apar când o instrucțiune are nevoie de rezultatele unei alte instrucțiuni care nu a fost încă finalizată. Ca potențiale soluții remarcăm:
   (a) **Data forwarding:** Rezultatele din etapele intermediare sunt "transmise înainte" pentru a fi utilizate fără a aștepta finalizarea completă.
   (b) **Stalling (blocare):** Pipeline-ul este temporar oprit pentru a aștepta disponibilitatea datelor.
   (c) **Out-of-order execution:** Un procesor OoO decuplează strict ordinea de preluare și decodificare a instrucțiunilor (care rămân in-order) de ordinea în care acestea sunt executate (care este out-of-order). Prin reorganizarea execuției instrucțiunilor, procesorul poate reduce latențele introduse de hazarduri. La final, instrucțiunile sunt menținute într-un buffer, iar rezultatele lor sunt finalizate și scrise în registre în ordinea originală (in-order commit).
2. **Hazard-uri de control** - Apar în cazul instrucțiunilor de salt (branching), când procesorul nu știe ce instrucțiune să preia până când condiția saltului este evaluată. Pentru a elimina din penalități putem folosi **Branch prediction** - prezicerea direcția saltului și începerea execuției

instrucțiunilor prezise. Dacă predicția este greșită, instrucțiunile incorecte din pipeline sunt eliminate.

3. **Hazard-uri structurale** - Apar când două sau mai multe instrucțiuni încearcă să folosească aceeași resursă hardware în același timp (ex. o singură unitate de acces la memorie). Pentru a evita acest aspect, este nevoie ca proiectarea procesorului să se realizeze cu resurse hardware redundante.

Exercițiu: Considerați următoarea declarație în memorie `.long 200, 300, 400`. Analizați pe rând pipeline-ul pentru instrucțiunile `lw a0, 4(gp)`, `sw a0, 8(gp)` respectiv `beq a0, a1, 12`.

Să încercăm în continuare să observăm pipeline-ul când avem un cod cu mai multe instrucțiuni:

```
.data
 .long 100, 200, 300
.text
.global main
main:
   lw a0, 4(gp)
   lw a1, 0(gp)
   add a0, a0, a1
   sw a0, 8(gp)
```

Exercițiu: Care este rezultatul depus în memorie? Ce s-a întâmplat?

Să observăm de asemenea codul următor:

```
.data
 .long 100, 200, 300
.text
.global main
main:
   lw a0, 0(gp)
   lw a1, 4(gp)
   blt a0, a1, label

   sw a0, 8(gp)
   j final

label:
   sw a1, 8(gp)

final:
   li a7, 93
   li a0, 0
   ecall
```

Exercițiu: Care este rezultatul depus în memorie? Ce s-a întâmplat?

Pentru rezolvarea acestor probleme, sunt oferite două variante îmbunătățite de procesor care abordează una dintre următoarele metode:

1. **Pipeline stall** (5-Stage processor w/o forwarding unit) - sunt introduse `nop`-uri (stall-uri) în conducta de date până când sursele sunt updatate.

2. **Forwarding/Bypassing** (5-stage processor w/o hazard detection) - sunt transmise datele din ALU direct către următoarea instrucțiune înainte de a fi scrise.

În cazul acesta însă cea de-a doua variantă nu este în continuare de folos, fiind necesar totuși mai mulți cicli între accesări. Pentru eficiență și corectitudine, vom folosi deci procesorul care implementează ambele mecanisme (5-stage processor).

Exercițiu: Observați avantajul mecanismului de forwarding (5-Stage processor w/o forwarding unit vs 5-stage processor).

În plus, Ripes oferă și o schemă logică extinsă pentru un procesor superscalar - dual issue care permite intrarea simultană în pipeline a două instrucțiuni. În plus față de procesorul anterior, acesta are 6 stage-uri, adăugându-se etapa **Issue** în care instrucțiunile sunt distribuite către unitățile funcționale disponibile (ALU, unități de acces la memorie).

5.2 Măsurarea performanței

1. **Throughput** - Măsoară câte instrucțiuni pot fi finalizate pe unitatea de timp. (5-stage processor - o instrucțiune, 6-stage dual-issue processor - 2 instrucțiuni).

2. **Latența** - Timpul total necesar pentru a finaliza o singură instrucțiune (din Fetch până la Write Back - 5 vs 6).

3. **CPI (Cycles Per Instruction)** - numărul mediu de cicluri necesare pentru a finaliza o instrucțiune. Într-un procesor pipelined, CPI ideal = 1, dar în realitate poate fi mai mare din cauza hazard-urilor.

4. **IPC (Instructions Per Cycle)** - numărul efectiv de instrucțiuni executate pe ciclu de ceas. Un procesor superscalar (dual-issue sau quad-issue) poate avea un IPC mai mare de 1 ($IPC = 1/CPI$).

5. **Utilizarea pipeline-ului** - procentul de timp în care pipeline-ul este complet ocupat cu sarcini utile. Idle time (timp de inactivitate) înseamnă că resursele nu sunt utilizate eficient.

Exemplu: Un program execută un număr egal de instrucțiuni aritmetice, de accesare a memoriei și cu floating point. Știm că o instrucțiune aritmetică ia 4 cicli, una de accesare a memoriei 5, iar una care accesează unitatea de floating point 6. Cât este **CPI**? Dar **IPC**? **Observație:** Acest tip de calcul ignoră multe alte variabile.

Răspuns:

$$CPI = 0.33 * 4 + 0.33 * 5 + 0.33 * 6 = 4.95$$

$$IPC = 1/4.95 = 0.2020$$

6 Cache

Memoria cache este un tip de memorie rapidă utilizată pentru a crește performanța procesorului, reducând timpul necesar pentru a accesa datele frecvent utilizate din memoria principală (RAM). Cache-ul este amplasat de obicei în apropierea procesorului și joacă un rol crucial într-un sistem de calcul modern.

Cache-ul funcționează ca o memorie intermediară între procesor și memoria principală, stocând datele utilizate recent sau predicții de date ce vor fi necesare în viitor. Prin aceasta, el reduce latența accesului la memorie și sporește eficiența execuției instrucțiunilor.

Cache-ul exploatează principiile **localității spațiale** (datele din apropierea unei locații utilizate recent sunt probabil necesare în viitor) și **localității temporale** (datele accesate recent sunt probabil utilizate din nou în scurt timp).

Exercițiu: Considerăm un sistem de calcul pe 32 de biți care are un cache de 16 Kbytes. Considerăm că memoriile sunt împărțite în blocuri de 32 de bytes. Câte blocuri sunt posibile în memoria principală? Câte blocuri sunt în memoria cache?

Memoria cache este organizată în mai multe niveluri ierarhice, fiecare având caracteristici distincte (L0, L1, L2 etc.). Dispozitivele cu capabilități reduse pot să nu aibă deloc sau să aibă mai puține niveluri de memorie cache. Simulatorul din Ripes de exemplu are doar memorie cache L1 pe care o vom folosi pentru a descoperi principalele caracteristici ale unei astfel de memorii.

O să începem prin a selecta o memorie cu 32 de linii sau blocuri și cu o dimensiune a liniei de 4 word-uri (16 bytes), direct-mapped (prima variantă preset). Să urmărim mai întâi algoritmul pentru descoperirea corespondenței dintre o adresă din memoria principală și locația din cache.

Să considerăm codul de mai jos:

```
.data
 .long 0x10, 0x11, 0x12, 0x13
 .long 0x14, 0x15, 0x16, 0x17
 .long 0x18, 0x19, 0x20, 0x21
.text
.global main
main:
   lw a0, 20(gp)      #0x10000014  --> 0x15
   lw a1, 24(gp)      #0x10000018  --> 0x16
   add a0, a0, a1
   sw a0, 28(gp)      #0x1000001c  --> 0x17
   lw a1, 0(gp)       #0x10000010  --> 0x10
   add a0, a0, a1
   sw a0, 32(gp)      #0x10000020  --> 0x18

final:
   li a7, 93
   li a0, 0
   ecall
```

Exercițiu: Vom considera `gp = 0x10000000`, valoarea inițială indicată de Ripes. Observați poziționarea în memoria cache. Puteți deduce regula?

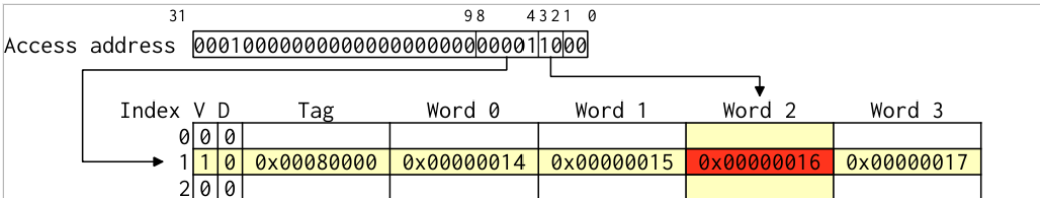


Figura 4: Linie de cache din Ripes

Algoritm

1. Unitatea de bază este word-ul, știm așadar ca ultimii 2 biți vor fi mereu 0 (adresele sunt din 4 în 4).
2. Extragerea offset-ului - Se determină câți biți sunt rezervați pentru offset pe baza dimensiunii blocului. În cazul nostru, o linie conține 16 bytes, 4 word-uri, deci offseții vor lua valori de la 0 la 3 - avem nevoie de încă 2 biți.
3. Extragerea indexului - Indexul indica linia/blocul din cache, deci va depinde de numărul de linii/blocuri din cache. În acest caz, avem 32 de linii, deci avem nevoie de 5 biți pentru identificare. Acesta este folosit pentru a identifica locația din cache în care s-ar putea afla adresa cerută din memorie.
4. Extragerea tag-ului - Tag-ul este reprezentat de restul de biți rămași. Tag-ul este comparat cu tag-ul stocat în acea locație din cache pentru a verifica dacă datele sunt cele căutate.

Exercițiu: Urmăriți cum se modifică memoria cache pe parcursul execuției programului. Care ar fi size-ul minim al unei linii pentru a avea cel mai bun hit rate?

Exercițiu: Care este corelația dintre dimensiunea cache-ului ca număr de linii, și dimensiunea unei linii pe de-o parte și conceptele de localitate temporală și spațială pe de cealaltă parte?

În plus față de aceste concepte, putem modifica și **tehnica de mapare**:

- **Mapare directă** - Fiecare bloc din memoria principală poate fi mapat doar într-o locație specifică din cache. Simplu de implementat, dar poate duce la conflicte frecvente.
- **Mapare asociativă** - Orice bloc din memoria principală poate fi mapat în orice locație din cache. Este mai flexibil, dar implică un cost mai mare pentru căutare.
- **Mapare x-way set asociativă** - Combinație între maparea directă și cea asociativă, unde cache-ul este împărțit în seturi, iar un bloc din memoria principală poate fi mapat într-un subset specific de locații din cache.

Exercițiu: Schimbați tipul de mapare în mapare asociativă. Ce observați?

Să considerăm acum următorul program:

```
.data
 .long 0x10, 0x11, 0x12, 0x13
 .long 0x14, 0x15, 0x16, 0x17
 .long 0x18, 0x19, 0x20, 0x21
.text
.global main
main:
   lw a0, 8(gp)
   lw a1, 12(gp)
   add a2, a0, a1
   lw a0, 24(gp)
   lw a1, 28(gp)
   add a3, a0, a1
   lw a0, 40(gp)
   lw a1, 44(gp)
   add a4, a0, a1
final:
   li a7, 93
   li a0, 0
   ecall
```

Exercițiu: Să considerăm acum o memorie cache 2-way set associative cu un line size de 2 și cu doar 2 linii de cache. Ce observați că se întâmplă?

Când cache-ul este plin, procesorul trebuie să decidă de asemenea ce bloc să înlocuiască. Politicile obișnuite includ:

- **LRU (Least Recently Used)** - Se înlocuiește blocul care nu a fost folosit de cel mai mult timp.
- **FIFO (First In First Out)** - Se înlocuiește blocul care a fost adăugat primul.
- **Random** - Blocul înlocuit este ales aleatoriu.

O altă caracteristică a cache-ului (din păcate nefuncțională pe Ripes) este politica de scriere:

1. **Write-through** - Procesorul modifică datele în blocul corespunzător din cache, iar modificarea este simultan scrisă și în memoria principală.
2. **Write-back** - Procesorul scrie datele doar în blocul din cache și setează un indicator numit bitul dirty (dirty bit). Dacă blocul trebuie înlocuit din cache, datele sunt scrise în memoria principală dacă acest bit este setat.

O altă politică adițională pentru scriere este **(No) Write Allocate** - decizia de a aduce blocul în memoria cache și la scriere sau nu.

6.1 Performanța memoriei cache

Performanța cache-ului este determinată de:

1. **Rata de accesare (hit rate)** - Procentajul accesărilor care găsesc datele necesare în cache. Rata ridicată de accesare reduce semnificativ latența memoriei.

2. **Latența memoriei** - Timpul necesar pentru a accesa datele. Latența este minimă pentru L1 și crește pentru L2, L3 și memoria principală.

3. **Penalty-ul ratării (miss penalty)** - Timpul suplimentar necesar pentru a obține datele din memoria principală atunci când acestea nu sunt prezente în cache.

4. **Politici de preluare (prefetching)** - Procesorul poate prelua proactiv date care ar putea fi necesare în viitor, bazându-se pe modele de acces..

7 Exercitii

7.1 Laboratorul 9

1. Se dau doua numere naturale x si y. Să se scrie un program care sa realizeze interschimbarea lor.

2. Se dau 4 numere a, b, c și d. Să se calculeze rezultatul operației $(((a + b + 5) * 2) \ll c)|d)$ și să se afișeze pe ecran.

3. Să se indice encodarea pentru instrucțiunea `and t0, s1, a5`.

7.2 Laboratorul 10

1. Afișați numărul de divizori ai unui întreg stocat în memorie și apoi o listă cu aceștia. ("Numărul 6 are 4 divizori: 1, 2, 3, 6").

2. Realizați o procedură `divizori(int x)` care realizează afișarea de mai sus și întoarce numărul de divizori.

3. Realizați o procedură `divizori_elemente(int *v, int n, int k)` care parcurge vectorul `v` cu `n` elemente și returnează numărul de elemente ce au exact `k` divizori. Afișați în `main` un mesaj de tipul "Sunt nr elemente cu exact k divizori". Folosiți-vă de procedura din exercițiul anterior.

7.3 Laboratorul 11

1. Un procesor are un pipeline cu 5 stadii: Fetch, Decode, Execute, Memory, Write-back. Determinați câte cicluri sunt necesare pentru a executa 4 instrucțiuni dacă nu apar hazarduri. Ce impact ar avea un hazard de date între instrucțiunile 1 și 2 asupra ciclurilor totale de execuție?

2. Să considerăm un procesor cu următoarea frecvență a instrucțiunilor și următoarele costuri:

   - ALU pentru întregi - 50%, 1 ciclu
   - Load - 20%, 5 cicli
   - Store - 10%, 1 ciclu
   - Branch - 20%, 2 cicli

   Calculați CPI și decideți care schimbare ar aduce cea mai bună performanță dintre următoarele:

   - implementarea unui mecanism de branch prediction pentru a reduce costul branch-urilor la 1 ciclu;
   - folosirea unui data cache care are reduce costul unui load la 3 cicli.

3. Dacă un procesor superscalar este introdus pentru a rezolva hazardurile de date, câte cicluri pot fi economisite pentru următoarea secvență de instrucțiuni:

```
ADD R1, R2, R3
SUB R4, R1, R5
MUL R6, R7, R8
```

4. Memoria principală are o dimensiune de 2^{16}, iar cache-ul are o capacitate de 1 KB, cu o dimensiune a blocului de 32 bytes. Calculați numărul total de blocuri din memoria principală. Calculați numărul de linii de cache. Determinați unde va fi mapată adresa 0x3A7F în cache.

5. Un cache asociativ setă cu 4 seturi și 2 linii per set utilizează politica LRU (Least Recently Used). Considerați următoarele adrese accesate: 0x0001, 0x0005, 0x0021, 0x0001, 0x0045, 0x0005. Explicați cum sunt organizate datele în cache după fiecare acces. Care bloc este înlocuit atunci când adresa 0x0045 este accesată?

8 Resurse suplimentare

- `https://riscv.org/educational-resources/`
- `https://www.allaboutcircuits.com/technical-articles/introductions-to-risc-v-instruction-set-understanding-this-open-instruction-set-architecture/`
- `https://www.cse.iitd.ac.in/~srsarangi/archbook/chapters/riscv.pdf`
- `https://passlab.github.io/CSE564/notes/lecture09_RISCV_Impl_pipeline.pdf`
- `https://nsec.sjtu.edu.cn/data/MK.Computer.Organization.and.Design.4th.Edition.Oct.2011.pdf`

	a7	**a0**	**Name**	**Description**
	1	(integer to print)	print_int	Prints the value located in a0 as a signed integer
	2	(float to print)	print_float	Prints the value located in a0 as a floating point number
	4	(pointer to string)	print_string	Prints the null-terminated string located at address in a0
	10	-	exit	Halts the simulator
	11	(char to print)	print_char	Prints the value located in a0 as an ASCII character
	34	(integer to print)	print_hex	Prints the value located in a0 as a hex number
	35	(integer to print)	print_bin	Prints the value located in a0 as a binary number
	36	(integer to print)	print_unsigned	Prints the value located in a0 as an unsigned integer
	93	(status code)	exit	Halts the simulator and exits with status code in a0